算法竞赛模板库

接口速查与源码

ExplodingKonjac

2026 年 11 月 4 日

目录

速查索引 **4**
第 1 章 基础 **5**
1.1 基础类型别名 5
1.1.1 接口 5
1.1.2 约定 5
1.1.3 完整源码 5
1.2 模运算 6
1.2.1 接口 6
1.2.2 约定 7
1.2.3 完整源码 7
1.3 数论 10
1.3.1 接口 10
1.3.2 约定 11
1.3.3 完整源码 11
第 2 章 数据结构 **16**
2.1 动态树 (Link-Cut Tree) 16
2.1.1 接口 16
2.1.2 约定 16
2.1.3 完整源码 17
2.2 配对堆 21
2.2.1 接口 21
2.2.2 约定 22
2.2.3 完整源码 22
第 3 章 字符串 **27**
3.1 后缀数组 27
3.1.1 接口 27
3.1.2 约定 27
3.1.3 完整源码 27
3.2 后缀自动机 29
3.2.1 接口 29
3.2.2 约定 30
3.2.3 完整源码 30
第 4 章 匹配 **33**
4.1 二分图最大匹配 33
4.1.1 接口 33
4.1.2 约定 33
4.1.3 完整源码 33
4.2 二分图最大权匹配 35
4.2.1 接口 35
4.2.2 约定 36
4.2.3 完整源码 36
4.3 一般图最大匹配 38
4.3.1 接口 38
4.3.2 约定 39
4.3.3 完整源码 39
4.4 一般图最大权完美匹配 41
4.4.1 接口 41
4.4.2 约定 42
4.4.3 完整源码 42
第 5 章 网络流 **52**
5.1 最大流 52
5.1.1 接口 52

5.1.2 约定 . . . 52
5.1.3 完整源码 . . . 52
5.2 最小费用流 . . . 55
5.2.1 接口 . . . 55
5.2.2 约定 . . . 55
5.2.3 完整源码 . . . 55
5.3 最小费用可行环流 . . . 60
5.3.1 接口 . . . 60
5.3.2 约定 . . . 60
5.3.3 完整源码 . . . 60
第 6 章 多项式 . . . 67
6.1 多项式运算 . . . 67
6.1.1 接口 . . . 67
6.1.2 约定 . . . 67
6.1.3 完整源码 . . . 68
6.2 多项式插值 . . . 76
6.2.1 接口 . . . 76
6.2.2 约定 . . . 76
6.2.3 完整源码 . . . 76
第 7 章 计算几何 . . . 79
7.1 计算几何基础 . . . 79
7.1.1 接口 . . . 79
7.1.2 约定 . . . 80
7.1.3 完整源码 . . . 80
7.2 圆 . . . 88
7.2.1 接口 . . . 88
7.2.2 约定 . . . 88
7.2.3 完整源码 . . . 89
7.3 多边形 . . . 93
7.3.1 接口 . . . 93
7.3.2 约定 . . . 93
7.3.3 完整源码 . . . 94
第 8 章 算法 . . . 100
8.1 点分治 . . . 100
8.1.1 待实现 . . . 101
8.1.2 数据契约 . . . 101
8.2 虚树 . . . 101
8.2.1 待实现 . . . 101
8.2.2 数据契约 . . . 102
8.3 Tarjan . . . 102
8.3.1 边双连通分量 . . . 102
8.3.2 点双连通分量 . . . 103
8.3.3 圆方树 . . . 104
8.3.4 数据契约 . . . 104
8.4 杜教筛 . . . 105
8.4.1 作用与适用范围 . . . 105
8.4.2 算法简述 . . . 105
8.4.3 伪代码 . . . 105
8.5 Min_25 筛 . . . 106
8.5.1 作用与适用范围 . . . 106
8.5.2 算法简述 . . . 106
8.5.3 伪代码 . . . 106
8.6 PN 筛 . . . 107
8.6.1 作用与适用范围 . . . 107
8.6.2 算法简述 . . . 108
8.6.3 伪代码 . . . 108

速查索引

下表按「基础 → 数据结构 → 字符串 → 匹配 → 网络流 → 多项式 → 几何 → 算法」分组，右列为各节起始页码；头文件列省略.hpp 后缀，——表示该节不对应头文件。

模块	头文件	复杂度	页
基础			
类型别名	def	——	5
模运算	modint	乘除 $O(\log p)$	6
数论	number_theory	分解期望 $O(n^{1/4})$	10
数据结构			
动态树	link_cut_tree	各项均摊 $O(\log n)$	16
配对堆	pairing_heap	合并 $O(1)$，删除均摊 $O(\log n)$	21
字符串			
后缀数组	suffix_array	$O(n \log n)$	27
后缀自动机	suffix_automaton	$O(n)$（空间 $O(nZ)$）	29
匹配			
二分图最大匹配	bipartite_matching	$O(E\sqrt{V})$	33
二分图最大权匹配	bipartite_weighted_matching	$O(n^2 m)$	35
一般图最大匹配	general_matching	$O(V^3)$	38
一般图最大权完美匹配	general_weighted_matching	$O(V^3)$	41
网络流			
最大流	max_flow	$O(V^2 E)$	52
最小费用流	min_cost_flow	$O(FE \log V)$	55
最小费用可行环流	min_cost_circulation	实际近似线性	60
多项式			
多项式运算	fpoly	$O(n \log n)$	67
多项式插值	interpolation	$O(n^2)$	76
计算几何			
计算几何基础	geometry	单次 $O(1)$	79
圆	geometry/circle	最小覆盖圆期望 $O(n)$	88
多边形	geometry/polygon	$O(n \log n)$	93
算法			
点分治	——	$O(n \log n)$	100
虚树	——	$O(k \log n)$	101
Tarjan	——	$O(n + m)$	102
杜教筛	——	$O(n^{2/3})$	105
Min_25 筛	——	$O(n^{3/4} / \log n)$	106
PN 筛	——	$O(n^{2/3})$	107

第 1 章 基础

1.1 基础类型别名

本模块为定长整数与常用浮点类型提供短别名，并暴露 GCC/Clang 的 128 位扩展类型；库中其余头文件都直接或间接依赖它，没有函数与类。

1.1.1 接口

接口	说明
`i8 / u8 / i16 / u16`	8 位、16 位有符号 / 无符号整数，对应 `std::int8_t` 等
`i32 / u32 / i64 / u64`	32 位、64 位有符号 / 无符号整数
`i128 / u128`	128 位整数，GNU 扩展（`__int128` / `unsigned __int128`）
`isize / usize`	指针宽度整数，等价于 `std::ptrdiff_t` / `std::size_t`
`f32 / f64`	单精度 / 双精度浮点，等价于 `float` / `double`

1.1.2 约定

别名全部位于命名空间 `acm`；除 `def.hpp` 外的头文件都以`"def.hpp"` 相对引入它，单独复制某个模块时需要一并复制 `def.hpp`。`usize` 就是 `std::size_t`，用 `printf` 输出时格式串写`%zu`，混用`%d` 在 64 位平台上会截断。`i128` 与 `u128` 是编译器扩展而非标准类型，MSVC 不支持，也没有对应的 `std::cin` / `std::cout` 重载。

1.1.3 完整源码

```cpp
#pragma once

#include <cstddef>
#include <cstdint>

namespace acm
{

using i8 = std::int8_t;
using u8 = std::uint8_t;
```

```cpp
using i16 = std::int16_t;
using u16 = std::uint16_t;
using i32 = std::int32_t;
using u32 = std::uint32_t;
using i64 = std::int64_t;
using u64 = std::uint64_t;
using i128 = __int128;
using u128 = unsigned __int128;
using isize = std::ptrdiff_t;
using usize = std::size_t;
using f32 = float;
using f64 = double;

}  // namespace acm
```

1.2 模运算

提供静态模数 SModint<P> 与动态模数 DModint 两种模整数，四则运算按模进行；另附快速幂、Legendre 符号与模平方根。

1.2.1 接口

接口	说明
SModint<P>	编译期模数；static_assert 要求 $0 < P < 2^{31}$，实际应取素数
DModint	运行期模数；set_mod(p) 修改全局模数，默认 998244353
Mint(x)	由任意整型构造，内部先 x 对模数取余再归约，负数也正确
mod()	返回当前模数；SModint 为 constexpr，DModint 为运行期
operator()	取出内部 i32 值；explicit operator bool 判断是否非零
+ - * / += -= *= /=	四则运算与复合赋值；除法通过乘逆元实现
== != ++ --	比较与自增自减；一元负号按模取反，语义与内置整数一致
inv()	返回乘法逆元；内部值必须非零，否则 assert 失败
acm::pow(a, b)	快速幂；b 为任意整型，为负时先对 a 求逆
acm::legendre(x)	返回 $0/1/-1$，要求模数为奇素数
acm::sqrt(a)	返回 std::optional<Mint>；无平方根时为空，要求模数为奇素数

1.2.2 约定

模数上限 2^{31} 是硬约束：乘法用 i64 中间量，模数再大会溢出。DModint::set_mod 修改的是全局静态变量，必须在构造任何 DModint 之前调用，同一进程内混用多个模数会互相污染。除法与 inv() 在除数为零时断言失败，关闭断言后是未定义行为；模数非素数时，扩展欧几里得只对 $\gcd(a,p)=1$ 的情形给出逆元。pow 的指数为负时先对底数求逆，因此底数不能为 0。

1.2.3 完整源码

```cpp
#pragma once

#include <cassert>
#include <concepts>
#include <optional>
#include <type_traits>
#include <utility>

#include "def.hpp"

namespace acm
{

namespace detail
{

template <typename D>
class ModintBase {
protected:
    i32 v = 0;

    constexpr static inline i32 qmo(i32 x) {
        return x + ((x >> 31) & D::mod());
    }
    constexpr operator D() const { return static_cast<const D&>(*this); }
    constexpr operator D&() { return static_cast<D&>(*this); }

public:
    constexpr ModintBase() = default;

    template <std::integral T>
    constexpr ModintBase(T x): v(qmo(x % D::mod())) {}

    constexpr i32 operator()() const { return v; }
```

```
35      constexpr explicit operator bool() const { return v != 0; }
36
37      friend constexpr bool operator==(D a, D b) { return a.v == b.v; }
38      friend constexpr bool operator!=(D a, D b) { return a.v != b.v; }
39
40      constexpr D& operator+=(D x) { return v = qmo(v - D::mod() + x.v), *this; }
41      constexpr D& operator-=(D x) { return v = qmo(v - x.v), *this; }
42      constexpr D& operator*=(D x) { return v = i64(v) * x.v % D::mod(), *this; }
43      constexpr D& operator/=(D x) { return *this *= x.inv(); }
44
45      friend constexpr D operator+(D a, D b) { return a += b; }
46      friend constexpr D operator-(D a, D b) { return a -= b; }
47      friend constexpr D operator*(D a, D b) { return a *= b; }
48      friend constexpr D operator/(D a, D b) { return a /= b; }
49
50      constexpr D operator+() const { return *this; }
51      constexpr D operator-() const { return D(0) - *this; }
52
53      constexpr D& operator++() { return *this += D(1); }
54      constexpr D& operator--() { return *this -= D(1); }
55      constexpr D operator++(int) {
56          D t = *this;
57          return ++*this, t;
58      }
59      constexpr D operator--(int) {
60          D t = *this;
61          return --*this, t;
62      }
63
64      constexpr D inv() const {
65          assert(v != 0);
66          i64 a = v, b = D::mod(), x = 1, y = 0;
67          while (b) {
68              i64 q = a / b;
69              std::swap(a -= q * b, b);
70              std::swap(x -= q * y, y);
71          }
72          return D(x < 0 ? x + D::mod() : x);
73      }
74  };
75
76  }  // namespace detail
```

```cpp

template <i32 P>
struct SModint: detail::ModintBase<SModint<P>> {
    static_assert(P > 0 && u32(P) < (1u << 31), "P must be in (0, 2^31)");

    using detail::ModintBase<SModint<P>>::ModintBase;
    static constexpr i32 mod() { return P; }
};

struct DModint: detail::ModintBase<DModint> {
    inline static i32 M = 998244353;

    using detail::ModintBase<DModint>::ModintBase;
    static void set_mod(i32 p) { assert(p > 0 && u32(p) < (1u << 31)), M = p; }
    static i32 mod() { return M; }
};

template <typename M, std::integral T>
constexpr M pow(M a, T b) {
    if constexpr (std::is_signed_v<T>)
        if (b < 0) return pow(a.inv(), -b);
    M r = 1;
    for (; b; b >>= 1, a *= a)
        if (b & 1) r *= a;
    return r;
}

template <typename M>
inline int legendre(M x) {
    i32 p = M::mod();
    if (x() == 0) return 0;
    if (p == 2) return 1;
    M r = pow(x, (p - 1) / 2);
    return r() == p - 1 ? -1 : 1;
}

template <typename M>
inline std::optional<M> sqrt(M a) {
    i32 p = M::mod();
    if (a() == 0) return a;
    if (p == 2) return a;
    if (legendre(a) != 1) return std::nullopt;
```

```
119     if (p % 4 == 3) return pow(a, (p + 1) / 4);
120
121     i32 q = p - 1;
122     u32 s = 0;
123     while (!(q & 1)) q >>= 1, s++;
124     M z = 2;
125     while (legendre(z) != -1) z += 1;
126     M c = pow(z, q), x = pow(a, (q + 1) / 2), t = pow(a, q);
127     for (u32 m = s; t != M(1);) {
128         u32 i = 1;
129         M y = t * t;
130         while (y != M(1)) y *= y, i++;
131         M b = pow(c, u64(1) << (m - i - 1));
132         x *= b;
133         c = b * b;
134         t *= c;
135         m = i;
136     }
137     return x;
138 }
139
140 }  // namespace acm
```

1.3 数论

模幂、扩展欧几里得、一次不定方程、判素与分解、模平方根，以及可组合的 Universal Euclidean Algorithm。

1.3.1 接口

接口	说明
`qpow<T>(a, b, p)`	返回 $a^b \bmod p$；要求 `sizeof(T) <= 8` 且 $b \geq 0$
`exgcd<T>(a, b, x, y)`	返回 $\gcd(a, b)$，并把一组解写入 `x`、`y`
`biequation<T>(a, b, c)`	解 $ax + by = c$，返回 x 最小非负的一组 `{x,y}`；无解返回 `{-1,-1}`
`is_prime(i64 n)`	Miller–Rabin 判素；$n < 2$ 返回 false，64 位内确定性正确
`pollard_rho(i64 n)`	返回 n 的一个非平凡因子；要求 n 为合数且 $n > 3$
`factorize(i64 n)`	返回升序质因子（含重数）；`n` 为 1 时返回空数组
`legendre(a, p)`	Legendre 符号 $\left(\frac{a}{p}\right) \in \{-1, 0, 1\}$，要求 p 为奇素数
`cipolla(n, p)`	返回 x 使 $x^2 \equiv n \pmod p$；无解返回 `std::nullopt`

接口	说明
`uniclidean<D,T,Op>(p, q, r, n, R, U, init, mul)`	返回格路径上各段按 `mul` 合并的结果

1.3.2 约定

`qpow` 的指数必须非负，负数会在移位循环中死循环，模数不要求素数。`biequation` 无解时返回 `{-1, -1}`，有解时返回 x 最小非负的一组解。`factorize` 只断言 $n > 0$：$n = 1$ 返回空数组，结果升序且含重数；`pollard_rho` 要求输入为合数，内部随机数种子固定，结果可复现。`legendre` 与 `cipolla` 要求 p 为奇素数，`cipolla` 返回其中一根，需要较小根时自行取 $\min(x, p - x)$。

1.3.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <cassert>
#include <concepts>
#include <cstdlib>
#include <functional>
#include <numeric>
#include <optional>
#include <random>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{

// Calculates a^b mod p
template <std::signed_integral T>
T qpow(T a, T b, T p) {
    static_assert(sizeof(T) <= 8, "qpow supports up to 64-bit integers");
    using W = i128;
    T r = 1;
    for (; b; b >>= 1, a = T(W(a) * a % p))
        if (b & 1) r = T(W(r) * a % p);
    return r;
}
```

```cpp

// Returns gcd(a,b) and assigned x,y to one solution of ax+by=gcd(a,b)
template <std::signed_integral T>
T exgcd(T a, T b, T& x, T& y) {
    if (!b) return x = 1, y = 0, a;
    T d = exgcd(b, a % b, y, x);
    return y -= a / b * x, d;
}

// Solve ax+by=c. returns {x,y}, making x>=0 and minimizing x.
template <std::signed_integral T>
std::pair<T, T> biequation(T a, T b, T c) {
    T x, y, g = exgcd(a, b, x, y);
    if (c % g) return {-1, -1};
    T dx = b / g, dy = a / g;
    i128 xx = x, yy = y, k = 0;
    c /= g, xx *= c, yy *= c;
    if (xx < 0) k = (-xx + dx - 1) / dx;
    if (xx > 0) k = -xx / dx;
    return {T(xx + k * dx), T(yy - k * dy)};
}

// Checks if n passes test_num Miller-Rabin tests.
inline bool is_prime(i64 n) {
    if (n < 2) return false;
    for (i64 p: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29}) {
        if (n % p == 0) return n == p;
    }
    i64 d = n - 1, s = 0;
    while (!(d & 1)) d >>= 1, s++;
    for (i64 a: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (a % n == 0) continue;
        i64 x = qpow(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (i64 i = 1; i < s; i++) {
            x = i64(i128(x) * x % n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
```

```
71          if (!ok) return false;
72      }
73      return true;
74  }
75
76  // Returns one of the non-trivial factor of n, using Pollard-Rho algorithm.
77  inline i64 pollard_rho(i64 n) {
78      if (n % 2 == 0) return 2;
79      if (n % 3 == 0) return 3;
80      static std::mt19937_64 rng(712367821);
81      std::uniform_int_distribution<i64> dist(0, n - 1);
82      while (true) {
83          i64 c = dist(rng) % (n - 1) + 1;
84          i64 x = dist(rng), y = x, d = 1;
85          auto f = [&](i64 v) { return i64((i128(v) * v + c) % n); };
86          while (d == 1) {
87              x = f(x);
88              y = f(f(y));
89              d = std::gcd(std::llabs(x - y), n);
90          }
91          if (d != n) return d;
92      }
93  }
94
95  // Factorize n and put the prime factors into res.
96  inline std::vector<i64> factorize(i64 n) {
97      assert(n > 0);
98      std::vector<i64> res, st{n};
99      while (!st.empty()) {
100         i64 x = st.back();
101         st.pop_back();
102         if (x == 1) continue;
103         if (is_prime(x)) {
104             res.push_back(x);
105             continue;
106         }
107         i64 p = pollard_rho(x);
108         st.push_back(p);
109         st.push_back(x / p);
110     }
111     std::sort(res.begin(), res.end());
112     return res;
```

```
}

// Returns the Legendre Symbol (a|p), assuming p is a odd prime number.
inline i64 legendre(i64 a, i64 p) {
    a %= p;
    if (a < 0) a += p;
    if (a == 0) return 0;
    if (p == 2) return 1;
    i64 r = qpow(a, (p - 1) / 2, p);
    return r == p - 1 ? -1 : 1;
}

// Returns x that x^2 mod p=n, or -1 if such x doesn't exists, assuming p is a
// odd prime number.
inline std::optional<i64> cipolla(i64 n, i64 p) {
    n %= p;
    if (n < 0) n += p;
    if (n == 0) return 0;
    if (p == 2) return n;
    if (legendre(n, p) != 1) return std::nullopt;

    static std::mt19937_64 rng(987654321);
    std::uniform_int_distribution<i64> dist(0, p - 1);
    i64 a = dist(rng);
    while (legendre(i64((i128(a) * a % p - n + p) % p), p) != -1) a = dist(rng);

    i64 i2 = i64((i128(a) * a % p - n + p) % p);
    using P = std::pair<i128, i128>;
    auto mul = [&](P x, P y) {
        return P{
            (x.first * y.first % p + i2 * x.second % p * y.second) % p,
            (x.first * y.second % p + x.second * y.first % p) % p,
        };
    };
    P base{a, 1}, res{1, 0};
    for (i64 k = (p + 1) / 2; k; k >>= 1, base = mul(base, base))
        if (k & 1) res = mul(res, base);
    return i64(res.first);
}

// Universal-Uclidean Algorithm. Computes the semigroup product of the segments:
// let f(x) = ⌊(p x + r) / q⌋, then
```

```cpp
//   (0, f(0)) -> (1, f(0)) -> ... -> (1, f(1)) -> ... ->
//   (i, f(i)) -> (i + 1, f(i)) -> ... -> (i + 1, f(i + 1)) -> ... ->
//   (n - 1, f(n - 1)) -> (n, f(n - 1)) -> ... -> (n, f(n))
// where going up denotes a `U` and going right denotes an `R`.
template <typename D, std::signed_integral T, typename Op = std::multiplies<D>>
D uniclidean(T p, T q, T r, T n, D R, D U, D init = {}, Op mul = Op{}) {
    if (n == 0) return init;
    auto qpow = [&](D x, T y, D init) {
        for (; y; y >>= 1, x = mul(x, x))
            if (y & 1) init = mul(init, x);
        return init;
    };
    if (r >= q) return uniclidean(p, q, r % q, n, R, U, init, mul);
    if (p >= q)
        return uniclidean(p % q, q, r, n, qpow(U, p / q, R), U, init, mul);
    T m = (p * n + r) / q;
    if (m == 0) return qpow(R, n, init);
    init = qpow(R, (q - r - 1) / p + 1, init);
    init = uniclidean(q, p, (q - r - 1) % p, m - 1, U, R, init, mul);
    return qpow(R, n - (q * m - r - 1) / p - 1, mul(init, U));
}

}  // namespace acm
```

第 2 章 数据结构

2.1 动态树（Link-Cut Tree）

用 Splay 维护动态森林的实链剖分，支持点权修改、路径聚合、加边、删边、显式换根与连通性查询。路径聚合要求运算满足结合律与交换律，不要求单位元。

2.1.1 接口

接口	说明
`LinkCutTree<T, Op>`	动态森林；点权类型 `T`，路径聚合运算 `Op`，默认 `std::plus<T>`
`LinkCutTree(const R& a, Op op = {})`	用范围 `a` 初始化点权；要求 `a.size()` 与下标访问
`LinkCutTree(usize n, T v, Op op = {})`	构造 n 个点，点权全部为 `v`
`usize size()`	点数 n
`bool modify(usize u, T v)`	把点 `u` 的权值改为 `v`；越界返回 `false`
`bool update(usize u, F f)`	用 `f` 变换点 `u` 的权值；越界返回 `false`
`std::optional<T> query(usize u, usize v)`	返回路径 $u \to v$ 的聚合值；越界或不在同一棵树时返回 `nullopt`
`usize find_root(usize u)`	返回 `u` 当前所在树的根；越界返回 `npos`
`bool connected(usize u, usize v)`	判断 `u` 与 `v` 是否连通；越界返回 `false`
`bool make_root(usize u)`	把 `u` 变成其所在树的根；越界返回 `false`
`bool link(usize u, usize v)`	加入边 `u-v`；越界、自环或已连通时返回 `false`
`bool cut(usize u, usize v)`	删除直接边 `u-v`；越界、自环或边不存在时返回 `false`
`npos`	表示「无」的哨兵值，等于 `usize(-1)`

2.1.2 约定

顶点编号为 $[0, n)$，路径聚合包含两个端点；`query(u, u)` 返回单点权值。`Op` 必须满足结合律与交换律，`plus`、`max`、`min`、`bit_xor`、`gcd` 等都可直接使用；实现只维护一个聚合值，不跟踪路径方向，因此不支持非交换路径积。由于不要求单位元，`pushup` 只合并非空儿子，`max/min` 不需要额外构造零元。

`query` 会先检查端点是否连通，因此断开时返回 `nullopt`，不会给出无意义的结果。`make_root(u)` 成功后，`u` 所在连通分量的当前根变为 `u`，该分量内所有点的 `find_root` 都返回 `u`，直到下一次 `make_root`、`query`、`link` 或 `cut` 再次改变根方向；`find_root` 只适合判断当前根或连通性，不能当作稳定标识。

link 拒绝自环与已连通端点，cut 只删除直接边；失败调用不改变逻辑森林，但可能改变内部有根方向。只支持点权；边权问题可把每条边拆成额外点后使用。所有操作均摊 $O(\log n)$，空间 $O(n)$。

2.1.3 完整源码

```cpp
#pragma once

#include <functional>
#include <optional>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{

template <typename T = i32, typename Op = std::plus<T>>
class LinkCutTree {
public:
    static constexpr usize npos = usize(-1);

private:
    struct Node {
        usize lc, rc, fa;
        bool rev;
        T val, sum;

        explicit Node(T v):
            lc(npos),
            rc(npos),
            fa(npos),
            rev(false),
            val(std::move(v)),
            sum(val) {}
    };

    usize n = 0;
    std::vector<Node> t;
    Op op{};

    bool islc(usize x) const {
        usize f = t[x].fa;
```

```
39          return f != npos && t[f].lc == x;
40      }
41      bool nroot(usize x) const {
42          usize f = t[x].fa;
43          return f == npos || (t[f].lc != x && t[f].rc != x);
44      }
45      void pushup(usize x) {
46          T s = t[x].val;
47          if (t[x].lc != npos) s = op(t[t[x].lc].sum, s);
48          if (t[x].rc != npos) s = op(s, t[t[x].rc].sum);
49          t[x].sum = std::move(s);
50      }
51      void pushrev(usize x) {
52          std::swap(t[x].lc, t[x].rc);
53          t[x].rev ^= 1;
54      }
55      void pushdown(usize x) {
56          if (!t[x].rev) return;
57          if (t[x].lc != npos) pushrev(t[x].lc);
58          if (t[x].rc != npos) pushrev(t[x].rc);
59          t[x].rev = false;
60      }
61      void pushall(usize x) {
62          if (!nroot(x)) pushall(t[x].fa);
63          pushdown(x);
64      }
65      void rotate(usize x) {
66          usize f = t[x].fa, g = t[f].fa;
67          if (islc(x)) {
68              t[f].lc = t[x].rc;
69              if (t[x].rc != npos) t[t[x].rc].fa = f;
70              t[x].rc = f;
71          } else {
72              t[f].rc = t[x].lc;
73              if (t[x].lc != npos) t[t[x].lc].fa = f;
74              t[x].lc = f;
75          }
76          if (!nroot(f)) (t[g].lc == f ? t[g].lc : t[g].rc) = x;
77          t[x].fa = g;
78          t[f].fa = x;
79          pushup(f);
80          pushup(x);
```

```
81      }
82      void splay(usize x) {
83          pushall(x);
84          for (; !nroot(x); rotate(x)) {
85              usize f = t[x].fa;
86              if (!nroot(f)) rotate(islc(f) == islc(x) ? f : x);
87          }
88      }
89      void access(usize x) {
90          for (usize y = npos; x != npos; x = t[y = x].fa) {
91              splay(x);
92              t[x].rc = y;
93              if (y != npos) t[y].fa = x;
94              pushup(x);
95          }
96      }
97      void makeroot(usize x) {
98          access(x);
99          splay(x);
100         pushrev(x);
101     }
102     usize findroot(usize x) {
103         access(x);
104         splay(x);
105         while (true) {
106             pushdown(x);
107             if (t[x].lc == npos) break;
108             x = t[x].lc;
109         }
110         splay(x);
111         return x;
112     }
113
114 public:
115     LinkCutTree() = default;
116
117     template <typename R>
118     explicit LinkCutTree(const R& a, Op op = {}): n(a.size()), t(), op(op) {
119         t.reserve(n);
120         for (usize i = 0; i < n; i++) t.emplace_back(a[i]);
121     }
122     LinkCutTree(usize n, T v, Op op = {}): n(n), t(), op(op) {
```

```
123             t.reserve(n);
124             for (usize i = 0; i < n; i++) t.emplace_back(v);
125         }
126
127         usize size() const { return n; }
128
129         bool modify(usize u, T v) {
130             if (u >= n) return false;
131             splay(u);
132             t[u].val = std::move(v);
133             pushup(u);
134             return true;
135         }
136         template <typename F>
137         bool update(usize u, F f) {
138             if (u >= n) return false;
139             splay(u);
140             t[u].val = f(t[u].val);
141             pushup(u);
142             return true;
143         }
144         std::optional<T> query(usize u, usize v) {
145             if (u >= n || v >= n) return std::nullopt;
146             makeroot(u);
147             if (findroot(v) != u) return std::nullopt;
148             access(v);
149             splay(v);
150             return t[v].sum;
151         }
152         usize find_root(usize u) { return u >= n ? npos : findroot(u); }
153         bool connected(usize u, usize v) {
154             if (u >= n || v >= n) return false;
155             if (u == v) return true;
156             return findroot(u) == findroot(v);
157         }
158         bool make_root(usize u) {
159             if (u >= n) return false;
160             makeroot(u);
161             return true;
162         }
163         bool link(usize u, usize v) {
164             if (u >= n || v >= n || u == v) return false;
```

```
165             makeroot(u);
166             if (findroot(v) == u) return false;
167             t[u].fa = v;
168             return true;
169         }
170         bool cut(usize u, usize v) {
171             if (u >= n || v >= n || u == v) return false;
172             makeroot(u);
173             access(v);
174             splay(v);
175             if (t[v].lc != u || t[u].rc != npos) return false;
176             t[v].lc = npos;
177             t[u].fa = npos;
178             pushup(v);
179             return true;
180         }
181     };
182
183     }  // namespace acm
```

2.2 配对堆

配对堆是一种可并堆：每个结点的所有儿子用一条兄弟链维护，合并时只需比较两个根，把较小者挂到较大者的儿子链首，因此合并、插入与取堆顶都是 $O(1)$，同时支持删除任意元素与修改键值。

2.2.1 接口

接口	说明
`PairingHeap<T, Compare>`	堆；元素类型 `T`，比较器 `Compare`，默认分别为 `i32` 与 `std::less<T>`
`point_iterator`	结点迭代器；`*it`、`it->` 访问元素，插入与合并不会使其失效
`point_iterator push(T v)`	插入 `v`，返回指向它的迭代器
`point_iterator emplace(Args&&... args)`	用参数原地构造元素并插入，返回其迭代器
`const T& top() const`	堆顶元素；空堆调用未定义
`T pop()`	弹出并返回堆顶；空堆调用未定义
`T erase(point_iterator it)`	删除 `it` 指向的元素并返回它
`void modify(point_iterator it, T v)`	把 `it` 指向的元素改为 `v`
`void join(PairingHeap& other)`	把 `other` 的元素全部并入，`other` 变空
`void clear()`	清空；所有迭代器失效

接口	说明
`usize size() const / bool empty() const`	元素个数 / 是否为空
拷贝构造、拷贝赋值	深复制整棵堆，副本与原件互不影响
移动构造、移动赋值	转移所有权，源堆变为空

2.2.2 约定

默认是**大根堆**：`Compare = std::less<T>` 时 `top()` 是最大元素，换成 `std::greater<>` 即得小根堆。`Compare` 须是严格弱序，`T` 须可构造且可赋值（`modify` 会赋值，`emplace` 会转发参数给 `T` 的构造函数）。

`push` / `emplace` 返回的 `point_iterator` 直接指向结点本身：插入、合并与修改其它元素都不会使它失效，只有该元素被 `pop`、`erase` 或堆被 `clear` 后才失效。`modify` 同时支持增大与减小键值：键值变大时保留子树，变小时把子树拆开重新合并，再把结点并回堆，因此 `it` 在修改后仍然有效，可以继续用于下一次 `modify` 或 `erase`。

`join(other)` 只改指针，$O(1)$ 完成；两个堆应使用相同的比较器，且不要传入自身。所有操作都不做空堆检查，`top`、`pop` 以及使用已失效的迭代器都是未定义行为。复杂度：`push`、`emplace`、`top`、`join` 为 $O(1)$，`pop`、`erase`、`modify` 均摊 $O(\log n)$，空间 $O(n)$。两遍合并、析构与拷贝都用递归遍历结点，代码更短；但递归深度与树高或兄弟链长度同阶，最坏可达 $O(n)$，深链很长（几十万级）时要留足栈空间。

2.2.3 完整源码

```cpp
#pragma once

#include <functional>
#include <utility>

#include "def.hpp"

namespace acm
{

template <typename T = i32, typename Compare = std::less<T>>
class PairingHeap {
    struct Node {
        T v;
        Node *s, *n, *p;

        template <typename... Args>
        explicit Node(Args&&... args):
            v(std::forward<Args>(args)...),
            s(nullptr),
```

```
21              n(nullptr),
22              p(nullptr) {}
23      };
24
25  public:
26      class point_iterator {
27          friend class PairingHeap;
28
29      public:
30          point_iterator() = default;
31          const T& operator*() const { return x->v; }
32          const T* operator->() const { return &x->v; }
33
34      private:
35          explicit point_iterator(Node* x): x(x) {}
36          Node* x = nullptr;
37      };
38
39  private:
40      Node* rt = nullptr;
41      usize sz = 0;
42      Compare cmp{};
43
44      // 把 y 挂到 x 的兄弟链头部，调用前保证 x 的键不小于 y。
45      Node* merge(Node* x, Node* y) const {
46          if (!x) return y;
47          if (!y) return x;
48          if (cmp(x->v, y->v)) std::swap(x, y);
49          if (x->s) x->s->p = y;
50          y->p = x;
51          y->n = x->s;
52          x->s = y;
53          return x;
54      }
55      // 两遍合并：先两两配对，再从右往左合并。
56      Node* merges(Node* x) const {
57          if (!x) return nullptr;
58          if (!x->n) {
59              x->p = nullptr;
60              return x;
61          }
62          Node* y = x->n;
```

```
63          Node* z = y->n;
64          x->p = y->p = nullptr;
65          x->n = y->n = nullptr;
66          return merge(merge(x, y), merges(z));
67      }
68      // 从兄弟链摘下 x: x 是根时清空 rt, 否则由父亲接替它的位置。
69      Node* detach(Node* x) {
70          Node* y = x->p;
71          if (!y) return std::exchange(rt, nullptr);
72          Node*& p = (y->n == x ? y->n : y->s);
73          x->p = nullptr;
74          p = std::exchange(x->n, nullptr);
75          if (p) p->p = y;
76          return x;
77      }
78      static void destroy(Node* x) {
79          if (!x) return;
80          destroy(x->s);
81          destroy(x->n);
82          delete x;
83      }
84      static Node* clone(Node* x, Node* p) {
85          if (!x) return nullptr;
86          Node* y = new Node(x->v);
87          y->p = p;
88          y->s = clone(x->s, y);
89          y->n = clone(x->n, p);
90          return y;
91      }
92
93  public:
94      PairingHeap() = default;
95      PairingHeap(const PairingHeap& o):
96          rt(clone(o.rt, nullptr)), sz(o.sz), cmp(o.cmp) {}
97      PairingHeap(PairingHeap&& o) noexcept:
98          rt(std::exchange(o.rt, nullptr)),
99          sz(std::exchange(o.sz, 0)),
100         cmp(std::move(o.cmp)) {}
101     PairingHeap& operator=(const PairingHeap& o) {
102         if (this != &o) {
103             destroy(rt);
104             rt = clone(o.rt, nullptr);
```

```
105             sz = o.sz;
106             cmp = o.cmp;
107         }
108         return *this;
109     }
110     PairingHeap& operator=(PairingHeap&& o) noexcept {
111         if (this != &o) {
112             destroy(rt);
113             rt = std::exchange(o.rt, nullptr);
114             sz = std::exchange(o.sz, 0);
115             cmp = std::move(o.cmp);
116         }
117         return *this;
118     }
119     ~PairingHeap() { destroy(rt); }
120 
121     template <typename... Args>
122     point_iterator emplace(Args&&... args) {
123         Node* x = new Node(std::forward<Args>(args)...);
124         rt = merge(rt, x);
125         sz++;
126         return point_iterator{x};
127     }
128     point_iterator push(T v) { return emplace(std::move(v)); }
129     void join(PairingHeap& o) {
130         rt = merge(rt, std::exchange(o.rt, nullptr));
131         sz += std::exchange(o.sz, 0);
132     }
133     void modify(point_iterator it, T v) {
134         Node* x = detach(it.x);
135         if (cmp(x->v, v)) {
136             x->v = std::move(v);
137         } else {
138             Node* y = merges(x->s);
139             x->v = std::move(v);
140             x->s = nullptr;
141             x = merge(x, y);
142         }
143         rt = merge(rt, x);
144     }
145     T pop() {
146         Node* x = rt;
```

```
147             rt = merges(x->s);
148             sz--;
149             T v = std::move(x->v);
150             delete x;
151             return v;
152         }
153         T erase(point_iterator it) {
154             Node* x = detach(it.x);
155             rt = merge(rt, merges(x->s));
156             sz--;
157             T v = std::move(x->v);
158             delete x;
159             return v;
160         }
161         void clear() {
162             destroy(rt);
163             rt = nullptr;
164             sz = 0;
165         }
166 
167         const T& top() const { return rt->v; }
168         usize size() const { return sz; }
169         bool empty() const { return !sz; }
170     };
171 
172     }  // namespace acm
```

第 3 章 字符串

3.1 后缀数组

对任意可随机访问、元素可比较的序列构造后缀数组，给出字典序排名、每个起点的排名以及相邻后缀的最长公共前缀。

3.1.1 接口

接口	说明
`SuffixArray(const R& s)`	构造；`R` 需支持 `size()` 与随机访问，元素之间可比较大小
`usize size()`	序列长度 n
`usize sa(usize i)`	字典序第 i 小的后缀的起始位置（$0 \le i < n$）
`usize rk(usize i)`	从位置 i 开始的后缀的排名，是 `sa` 的逆映射
`usize height(usize i)`	第 i 小与第 $i-1$ 小后缀的 LCP；`height(0) == 0`

3.1.2 约定

元素只需支持 `<` 与`!=`，不必是字符；整数序列、`std::string` 与 `std::vector<int>` 都可直接传入。空序列合法，但此时不要调用 `sa`、`rk`、`height`。`sa` 与 `rk` 互为逆映射，下标均从 0 开始；`height(0)` 没有「前一个后缀」，按定义为 0，求任意两后缀的 LCP 需要另做 RMQ。

3.1.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <numeric>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{
```

```cpp

class SuffixArray {
private:
    std::vector<usize> a, b, h;

public:
    template <typename R>
    explicit SuffixArray(const R& s) {
        usize n = s.size();
        a.resize(n);
        b.resize(n);
        h.assign(n, 0);
        if (!n) return;

        std::iota(a.begin(), a.end(), usize(0));
        std::sort(a.begin(), a.end(), [&](usize x, usize y) {
            return s[x] < s[y];
        });

        usize m = 1;
        for (usize i = 0; i < n; i++) {
            if (i && s[a[i - 1]] != s[a[i]]) m++;
            b[a[i]] = m - 1;
        }

        std::vector<usize> old(n), tmp(n), bk(n);
        for (usize w = 1; m < n; w <<= 1) {
            if (w >= n) break;
            auto eq = [&](usize x, usize y) {
                return old[x] == old[y] &&
                    ((x + w < n && y + w < n && old[x + w] == old[y + w]) ||
                     (x + w >= n && y + w >= n));
            };
            usize tot = 0;
            for (usize i = n - w; i < n; i++) tmp[tot++] = i;
            for (usize i = 0; i < n; i++)
                if (a[i] >= w) tmp[tot++] = a[i] - w;
            bk.assign(m, 0);
            for (usize i = 0; i < n; i++) bk[b[i]]++;
            for (usize i = 1; i < m; i++) bk[i] += bk[i - 1];
            for (usize i = n; i > 0; i--) a[--bk[b[tmp[i - 1]]]] = tmp[i - 1];
            old.swap(b);
```

```
54              m = 1;
55              for (usize i = 0; i < n; i++) {
56                  if (i && !eq(a[i], a[i - 1])) m++;
57                  b[a[i]] = m - 1;
58              }
59          }
60
61          for (usize i = 0, j = 0; i < n; i++) {
62              if (j) j--;
63              if (b[i] == 0) continue;
64              usize k = a[b[i] - 1];
65              while (i + j < n && k + j < n && s[i + j] == s[k + j]) j++;
66              h[b[i]] = j;
67          }
68      }
69
70      usize size() const { return a.size(); }
71      usize sa(usize i) const { return a[i]; }
72      usize rk(usize i) const { return b[i]; }
73      usize height(usize i) const { return h[i]; }
74  };
75
76  }  // namespace acm
```

3.2 后缀自动机

逐字符插入并维护字符串的后缀自动机，可判断任意串是否为子串，也可遍历状态转移；转移用定长数组稠密存储，适合字符集很小的情形。

3.2.1 接口

接口	说明
`SuffixAutomaton<Symbol, Z>`	建一个只含初始状态 0 的空自动机；`Z` 为字符集大小
`usize extend(Symbol c)`	插入一个符号，返回新状态编号；同时更新 `last()`
`usize size()`	当前状态数，编号范围 $[0, \text{size})$
`usize last()`	当前整个串对应的状态
`usize link(usize i)`	后缀链接；初始状态 0 的链接为 `npos`
`usize max_len(usize i)`	该状态代表的最长串长度
`const std::array<usize, Z>& transitions(usize i)`	该状态的转移表，可直接遍历

接口	说明
`std::optional<usize> transition(usize i, Symbol c)`	走一条转移；不存在时返回 `nullopt`
`npos`	表示「无」的哨兵值，等于 `usize(-1)`

3.2.2 约定

模板参数 `Symbol` 只决定参数类型，`extend` 与 `transition` 直接把参数当转移下标，因此必须传 $0 \dots Z-1$ 的编号（例如 `c - 'a'`），传字符本身会越界。`Z` 是字符集大小，转移为定长稠密数组，空间 $O(|V|Z)$；字符集大时应改用 `std::map` 版本，本模块不提供。状态 0 为初始状态，`link(0)` 为 `npos`；`extend` 返回新状态编号并更新 `last()`，克隆产生的新状态也会计入 `size()`。

3.2.3 完整源码

```cpp
#pragma once

#include <array>
#include <optional>
#include <vector>

#include "def.hpp"

namespace acm
{

template <typename Symbol, usize Z>
class SuffixAutomaton {
    static_assert(Z > 0);

    std::vector<std::array<usize, Z>> nxt;
    std::vector<usize> fa, len;
    usize lst = 0;

public:
    static constexpr usize npos = usize(-1);

    SuffixAutomaton() {
        std::array<usize, Z> e;
        e.fill(npos);
        nxt.push_back(e);
        fa.push_back(npos);
```

```
28          len.push_back(0);
29      }
30
31      usize size() const { return len.size(); }
32      usize last() const { return lst; }
33      usize link(usize i) const { return fa[i]; }
34      usize max_len(usize i) const { return len[i]; }
35      const std::array<usize, Z>& transitions(usize i) const { return nxt[i]; }
36
37      std::optional<usize> transition(usize i, Symbol c) const {
38          usize to = nxt[i][usize(c)];
39          return to == npos ? std::nullopt : std::optional<usize>{to};
40      }
41
42      usize extend(Symbol c) {
43          usize x = usize(c), cur = size(), p = lst;
44          std::array<usize, Z> e;
45          e.fill(npos);
46          nxt.push_back(e);
47          fa.push_back(0);
48          len.push_back(len[p] + 1);
49          lst = cur;
50          for (; p != npos && nxt[p][x] == npos; p = fa[p]) nxt[p][x] = cur;
51          if (p == npos) {
52              fa[cur] = 0;
53          } else {
54              usize q = nxt[p][x];
55              if (len[p] + 1 == len[q]) {
56                  fa[cur] = q;
57              } else {
58                  usize nq = size();
59                  nxt.push_back(nxt[q]);
60                  fa.push_back(fa[q]);
61                  len.push_back(len[p] + 1);
62                  for (; p != npos && nxt[p][x] == q; p = fa[p]) nxt[p][x] = nq;
63                  fa[q] = nq;
64                  fa[cur] = nq;
65              }
66          }
67          return cur;
68      }
69  };
```

```
70
71  }  // namespace acm
```

第 4 章 匹配

4.1 二分图最大匹配

左右部分别编号的二分图，求最大匹配，返回匹配数与每个左部点的匹配对象。

4.1.1 接口

接口	说明
`BipartiteMatching(usize n, usize m)`	建左右部分别为 n、m 个点的空图，编号均为 $[0,n)$ / $[0,m)$
`usize size_l() / size_r()`	左部 / 右部点数
`void add_edge(usize u, usize v)`	加入左部 u 到右部 v 的边；允许重边，无自环概念
`std::pair<usize, std::vector<usize>> max_matching()`	返回 `{size, match}`（匹配数与左部匹配表）；`match[u]` 为左部 u 匹配的右部点，未匹配时为 `npos`
`npos`	表示「未匹配」的哨兵，等于 `usize(-1)`

4.1.2 约定

左右部各自从 0 开始独立编号，不要把右部点编到 $[n, n+m)$。`max_matching` 返回 `{size, match}`，`match[u]` 为左部 u 匹配的右部点，未匹配时为 `npos`。求解状态全部是函数内的局部变量，可以重复调用；两次调用之间新增的边会在下一次求解中计入。允许重边，重边会重复计入 E。

4.1.3 完整源码

```cpp
#pragma once

#include <queue>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{

```

```cpp
class BipartiteMatching {
    usize n, m;
    std::vector<std::vector<usize>> g;

public:
    static constexpr usize npos = usize(-1);

    BipartiteMatching(usize n, usize m): n(n), m(m), g(n) {}

    usize size_l() const { return n; }
    usize size_r() const { return m; }
    void add_edge(usize u, usize v) { g[u].push_back(v); }

    std::pair<usize, std::vector<usize>> max_matching() {
        std::vector<usize> ml(n, npos), mr(m, npos), dis(n);
        usize ans = 0, shortest = npos;

        auto bfs = [&] {
            std::queue<usize> q;
            shortest = npos;
            for (usize u = 0; u < n; u++) {
                if (ml[u] == npos) dis[u] = 0, q.push(u);
                else dis[u] = npos;
            }
            while (!q.empty()) {
                usize u = q.front();
                q.pop();
                if (dis[u] + 1 >= shortest) continue;
                for (usize v: g[u]) {
                    usize w = mr[v];
                    if (w == npos) shortest = dis[u] + 1;
                    else if (dis[w] == npos) {
                        dis[w] = dis[u] + 1;
                        q.push(w);
                    }
                }
            }
            return shortest != npos;
        };

        auto dfs = [&](auto&& self, usize u) -> bool {
            for (usize v: g[u]) {
```

```
                usize w = mr[v];
                if ((w == npos && dis[u] + 1 == shortest) ||
                    (w != npos && dis[w] == dis[u] + 1 && self(self, w))) {
                    ml[u] = v;
                    mr[v] = u;
                    return true;
                }
            }
            dis[u] = npos;
            return false;
        };

        while (bfs())
            for (usize u = 0; u < n; u++)
                if (ml[u] == npos && dfs(dfs, u)) ans++;
        return {ans, ml};
    }
};

}  // namespace acm
```

4.2 二分图最大权匹配

把每个左部点匹配到不同的右部点并使总权最大，边权可为负；不存在覆盖全部左部点的匹配时返回空。

4.2.1 接口

接口	说明
`BipartiteWeightedMatching<T>`	模板参数须为有符号整数或浮点（`static_assert` 检查）
`BipartiteWeightedMatching(usize n, usize m)`	建左右部各 n、m 个点的完全空图，编号从 0 开始
`usize size_l() / size_r()`	左部 / 右部点数
`void add_edge(u, v, T x)`	加边；重边保留较大权值，等价于 `set_edge(u,v,max(old,x))`
`void set_edge(u, v, T x)`	直接覆盖边权，不做取大
`T get_edge(u, v)`	返回当前边权；无边时返回 `no_edge`
`max_weighted_matching()`	返回 `std::optional<std::pair<T, std::vector<usize>>>`：最大权与 `match[u]`；无解时为空

接口	说明
no_edge	「无边」哨兵，等于 std::numeric_limits<T>::lowest()，不能作为有效边权
npos	表示「无」的哨兵，等于 usize(-1)

4.2.2 约定

必须 $n \le m$，否则直接返回 nullopt；每个左部点至少要有一条边，否则同样无解。no_edge 是 lowest()，不能作为有效边权。add_edge 对重边保留较大值，set_edge 直接覆盖，get_edge 查当前权值。返回 {weight, match}，match 长度等于 n，无解时整体为空；最终权值和必须能由 T 表示。

4.2.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <limits>
#include <optional>
#include <type_traits>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{

template <typename T>
class BipartiteWeightedMatching {
    static_assert(std::is_signed_v<T> || std::is_floating_point_v<T>);

    using Calc =
        std::conditional_t<std::is_floating_point_v<T>, long double, i128>;

    static constexpr Calc inf() {
        if constexpr (std::is_floating_point_v<T>)
            return std::numeric_limits<Calc>::infinity();
        else return Calc(1) << 120;
    }
    static constexpr Calc INF = inf();

```

```
29      usize n, m;
30      std::vector<std::vector<T>> w;
31
32  public:
33      static constexpr usize npos = usize(-1);
34      static constexpr T no_edge = std::numeric_limits<T>::lowest();
35
36      BipartiteWeightedMatching(usize n, usize m):
37          n(n), m(m), w(n, std::vector<T>(m, no_edge)) {}
38
39      usize size_l() const { return n; }
40      usize size_r() const { return m; }
41
42      void add_edge(usize u, usize v, T x) { w[u][v] = std::max(w[u][v], x); }
43      void set_edge(usize u, usize v, T x) { w[u][v] = x; }
44      T get_edge(usize u, usize v) const { return w[u][v]; }
45
46      std::optional<std::pair<T, std::vector<usize>>> max_weighted_matching() {
47          if (n > m) return std::nullopt;
48
49          std::vector<usize> p(m + 1), way(m + 1);
50          std::vector<Calc> lx(n + 1), ly(m + 1);
51          for (usize i = 1; i <= n; i++) {
52              lx[i] = Calc(no_edge);
53              for (usize j = 1; j <= m; j++)
54                  if (w[i - 1][j - 1] != no_edge)
55                      lx[i] = std::max(lx[i], Calc(w[i - 1][j - 1]));
56              if (lx[i] == Calc(no_edge)) return std::nullopt;
57          }
58
59          std::vector<Calc> slack;
60          std::vector<u8> used;
61          for (usize s = 1; s <= n; s++) {
62              p[0] = s;
63              slack.assign(m + 1, INF);
64              used.assign(m + 1, false);
65              usize u = 0;
66              do {
67                  usize i = p[u], v = npos;
68                  Calc delta = INF;
69                  used[u] = true;
70                  for (usize j = 1; j <= m; j++) {
```

```
                    if (used[j]) continue;
                    if (w[i - 1][j - 1] != no_edge) {
                        Calc cur = lx[i] + ly[j] - Calc(w[i - 1][j - 1]);
                        if (cur < slack[j]) slack[j] = cur, way[j] = u;
                    }
                    if (slack[j] < delta) delta = slack[j], v = j;
                }
                if (delta == INF) return std::nullopt;
                for (usize j = 0; j <= m; j++) {
                    if (used[j]) lx[p[j]] -= delta, ly[j] += delta;
                    else if (j && slack[j] != INF) slack[j] -= delta;
                }
                u = v;
            } while (p[u] != 0);
            do {
                usize v = way[u];
                p[u] = p[v];
                u = v;
            } while (u);
        }

        T result{};
        std::vector<usize> match(n);
        for (usize j = 1; j <= m; j++) {
            if (!p[j]) continue;
            usize i = p[j] - 1;
            match[i] = j - 1;
            result += w[i][j - 1];
        }
        return std::pair<T, std::vector<usize>>{result, std::move(match)};
    }
};

}  // namespace acm
```

4.3 一般图最大匹配

无向一般图的最大匹配。奇数长度的环无法用二分图方法处理，实现中使用 Edmonds Blossom 缩花。

4.3.1 接口

接口	说明
`GeneralMatching(usize n)`	建 n 个点、零条边的无向图，编号 $[0, n)$
`usize size()`	点数
`void add_edge(usize u, usize v)`	加入无向边；内部双向存储，端点应不同
`std::pair<usize, std::vector<usize>> max_matching()`	返回 `{size, match}`（匹配数与匹配表）；`match[u]` 为 u 的匹配点，未匹配时为 `npos`
`npos`	表示「未匹配」的哨兵，等于 `usize(-1)`

4.3.2 约定

每条无向边只需 `add_edge` 一次，内部会同时加入两个方向；重复添加不影响正确性，但会增大常数。端点必须不同，自环在匹配中没有意义，实现也未专门处理。返回 `{size, match}`，`match` 是对称的，未匹配为 `npos`；每次调用都从零开始求解，状态不跨调用保留。

4.3.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <numeric>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{

class GeneralMatching {
    usize n;
    std::vector<std::vector<usize>> g;

public:
    static constexpr usize npos = usize(-1);

    explicit GeneralMatching(usize n): n(n), g(n) {}
    usize size() const { return n; }

    void add_edge(usize u, usize v) {
        g[u].push_back(v);
```

```
25          g[v].push_back(u);
26      }
27
28      std::pair<usize, std::vector<usize>> max_matching() {
29          std::vector<usize> match(n, npos), fa(n), base(n), q;
30          std::vector<u8> used(n), bl(n);
31
32          auto lca = [&](usize u, usize v) {
33              std::vector<u8> seen(n);
34              while (true) {
35                  u = base[u];
36                  seen[u] = true;
37                  if (match[u] == npos) break;
38                  u = fa[match[u]];
39              }
40              while (!seen[base[v]]) v = fa[match[base[v]]];
41              return base[v];
42          };
43          auto mark = [&](usize u, usize b, usize child) {
44              while (base[u] != b) {
45                  bl[base[u]] = bl[base[match[u]]] = true;
46                  fa[u] = child;
47                  child = match[u];
48                  u = fa[match[u]];
49              }
50          };
51          auto find = [&](usize root) {
52              std::iota(base.begin(), base.end(), usize(0));
53              std::fill(used.begin(), used.end(), false);
54              std::fill(fa.begin(), fa.end(), npos);
55              q.clear();
56              q.push_back(root);
57              used[root] = true;
58              for (usize h = 0; h < q.size(); h++) {
59                  usize u = q[h];
60                  for (usize v: g[u]) {
61                      if (base[u] == base[v] || match[u] == v) continue;
62                      if (v == root ||
63                          (match[v] != npos && fa[match[v]] != npos)) {
64                          usize b = lca(u, v);
65                          std::fill(bl.begin(), bl.end(), false);
66                          mark(u, b, v);
```

```
                        mark(v, b, u);
                        for (usize x = 0; x < n; x++) {
                            if (!bl[base[x]]) continue;
                            base[x] = b;
                            if (!used[x]) used[x] = true, q.push_back(x);
                        }
                    } else if (fa[v] == npos) {
                        fa[v] = u;
                        if (match[v] == npos) return v;
                        v = match[v];
                        used[v] = true;
                        q.push_back(v);
                    }
                }
            }
            return npos;
        };

        usize ans = 0;
        for (usize root = 0; root < n; root++) {
            if (match[root] != npos) continue;
            usize v = find(root);
            if (v == npos) continue;
            ans++;
            while (v != npos) {
                usize u = fa[v], nxt = match[u];
                match[v] = u;
                match[u] = v;
                v = nxt;
            }
        }
        return {ans, std::move(match)};
    }
};

}  // namespace acm
```

4.4 一般图最大权完美匹配

在无向一般图中求覆盖所有顶点且边权和最大的匹配，支持负权边与浮点边权。

4.4.1 接口

接口	说明
`GeneralWeightedMatching<T>`	模板参数须为有符号整数或浮点（`requires` 约束）
`GeneralWeightedMatching(usize n)`	建 n 个点的空图，编号 $[0,n)$，无向
`usize size()`	点数
`void add_edge(u, v, T w)`	加无向边；重边保留较大权值
`void set_edge(u, v, T w)`	直接覆盖边权（两个方向同时写入）
`T get_edge(u, v)`	当前边权；无边时为 `no_edge`
`max_weighted_matching()`	返回 `std::optional<std::pair<T, std::vector<usize>>>`：最大权与 `match`；无完美匹配时为空
`no_edge`	「无边」哨兵，等于 `std::numeric_limits<T>::lowest()`，不可作有效边权
`npos`	表示「无」的哨兵，等于 `usize(-1)`

4.4.2 约定

顶点数为奇数时直接返回 `nullopt`，顶点数为 0 时返回 `{0, {}}`。返回前会逐个顶点检查：只要有一个顶点未匹配或匹配边不存在就返回 `nullopt`，因此成功返回的一定是完美匹配，`match` 中不会出现 `npos`。浮点边权必须是有限值，`no_edge` 不能作为有效边权；最终权值和仍须由 `T` 表示。

4.4.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <cmath>
#include <concepts>
#include <limits>
#include <optional>
#include <type_traits>
#include <utility>
#include <vector>

#include "def.hpp"

namespace acm
{

template <typename T>
    requires(std::signed_integral<T> || std::floating_point<T>)
```

```cpp
class GeneralWeightedMatching {
private:
    using Calc = std::conditional_t<
        std::floating_point<T>,
        long double,
        std::conditional_t<(sizeof(T) < 8), i64, i128>
    >;

    struct Edge {
        usize u{}, v{};
        Calc w{};
    };

    usize N;
    std::vector<std::vector<T>> W;

public:
    static constexpr usize npos = -1;
    static constexpr T no_edge = std::numeric_limits<T>::lowest();

    explicit GeneralWeightedMatching(usize n):
        N{n}, W(n, std::vector<T>(n, no_edge)) {}

    usize size() const { return N; }

    void add_edge(usize u, usize v, T w) {
        W[u][v] = W[v][u] = std::max(W[u][v], w);
    }

    void set_edge(usize u, usize v, T w) { W[u][v] = W[v][u] = w; }
    T get_edge(usize u, usize v) const { return W[u][v]; }

    std::optional<std::pair<T, std::vector<usize>>> max_weighted_matching() {
        if (N % 2) return std::nullopt;
        if (!N) return std::pair<T, std::vector<usize>>{T{}, {}};

        const usize n = N;
        usize node_count = n, visit_id = 0;
        Calc eps{};
        std::vector<std::vector<Edge>> g(
            2 * n + 1, std::vector<Edge>(2 * n + 1)
        );
```

```
61          std::vector<usize> match(2 * n + 1), slack(2 * n + 1);
62          std::vector<usize> rep(2 * n + 1), par(2 * n + 1);
63          std::vector<usize> visit(2 * n + 1);
64          // Alternating-forest state: -1 outside, 0 outer, 1 inner.
65          std::vector<i32> state(2 * n + 1);
66          std::vector<Calc> label(2 * n + 1);
67          std::vector<std::vector<usize>> from(
68              2 * n + 1, std::vector<usize>(n + 1)
69          );
70          std::vector<std::vector<usize>> blossom(2 * n + 1);
71          std::vector<usize> queue;
72          usize queue_head = 0;
73
74          bool has_edge = false;
75          Calc minimum{}, maximum{};
76          for (usize u = 0; u < n; ++u) {
77              for (usize v = u + 1; v < n; ++v) {
78                  if (W[u][v] == no_edge) continue;
79                  Calc w = static_cast<Calc>(W[u][v]);
80                  if (!has_edge) minimum = maximum = w;
81                  else {
82                      minimum = std::min(minimum, w);
83                      maximum = std::max(maximum, w);
84                  }
85                  has_edge = true;
86              }
87          }
88          Calc range = has_edge ? maximum - minimum : 0;
89          // One more matched edge must dominate every possible weight gain.
90          Calc bonus = (range + 1) * static_cast<Calc>(n / 2 + 1);
91          Calc largest = 0;
92          for (usize u = 1; u <= n; ++u) {
93              rep[u] = u;
94              from[u][u] = u;
95              for (usize v = u + 1; v <= n; ++v) {
96                  if (W[u - 1][v - 1] == no_edge) continue;
97                  Calc transformed =
98                      static_cast<Calc>(W[u - 1][v - 1]) - minimum + bonus;
99                  g[u][v] = {u, v, transformed};
100                 g[v][u] = {v, u, transformed};
101                 largest = std::max(largest, transformed);
102             }
```

```
103         }
104         std::fill(label.begin() + 1, label.begin() + n + 1, largest);
105         if constexpr (std::floating_point<T>) {
106             eps = static_cast<Calc>(std::numeric_limits<T>::epsilon()) *
107                 64 *
108                 std::max<Calc>(1, largest);
109         }
110 
111         auto delta = [&](const Edge& edge) {
112             return label[edge.u] + label[edge.v] - edge.w * 2;
113         };
114         auto less = [&](Calc a, Calc b) {
115             if constexpr (std::floating_point<T>) return a < b - eps;
116             else return a < b;
117         };
118         auto zero = [&](Calc x) {
119             if constexpr (std::floating_point<T>) return std::abs(x) <= eps;
120             else return x == 0;
121         };
122         auto normalize = [&](Calc& x) {
123             if constexpr (std::floating_point<T>) {
124                 if (zero(x)) x = 0;
125             }
126         };
127         auto update_slack = [&](usize u, usize x) {
128             if (!slack[x] || less(delta(g[u][x]), delta(g[slack[x]][x])))
129                 slack[x] = u;
130         };
131         auto set_slack = [&](usize x) {
132             slack[x] = 0;
133             for (usize u = 1; u <= n; ++u) {
134                 if (g[u][x].w > 0 && rep[u] != x && state[rep[u]] == 0)
135                     update_slack(u, x);
136             }
137         };
138         auto queue_push = [&](auto&& self, usize x) -> void {
139             if (x <= n) queue.push_back(x);
140             else
141                 for (usize v: blossom[x]) self(self, v);
142         };
143         auto set_representative = [&](auto&& self, usize x, usize b) -> void {
144             rep[x] = b;
```

```
145             if (x > n)
146                 for (usize v: blossom[x]) self(self, v, b);
147         };
148         auto rotate_blossom = [&](usize b, usize entry) {
149             usize pos = std::find(blossom[b].begin(), blossom[b].end(), entry) -
150                 blossom[b].begin();
151             if (pos % 2) {
152                 std::reverse(blossom[b].begin() + 1, blossom[b].end());
153                 return blossom[b].size() - pos;
154             }
155             return pos;
156         };
157         auto set_match = [&](auto&& self, usize u, usize v) -> void {
158             match[u] = g[u][v].v;
159             if (u <= n) return;
160             usize entry = from[u][g[u][v].u];
161             usize pos = rotate_blossom(u, entry);
162             for (usize i = 0; i < pos; ++i)
163                 self(self, blossom[u][i], blossom[u][i ^ 1]);
164             self(self, entry, v);
165             std::rotate(
166                 blossom[u].begin(), blossom[u].begin() + pos, blossom[u].end()
167             );
168         };
169         auto augment = [&](usize u, usize v) {
170             while (true) {
171                 usize next = rep[match[u]];
172                 set_match(set_match, u, v);
173                 if (!next) return;
174                 set_match(set_match, next, rep[par[next]]);
175                 u = rep[par[next]];
176                 v = next;
177             }
178         };
179         auto lca = [&](usize u, usize v) {
180             ++visit_id;
181             for (; u || v; std::swap(u, v)) {
182                 if (!u) continue;
183                 if (visit[u] == visit_id) return u;
184                 visit[u] = visit_id;
185                 u = rep[match[u]];
186                 if (u) u = rep[par[u]];
```

```
187             }
188             return usize{0};
189         };
190         auto add_blossom = [&](usize u, usize base, usize v) {
191             usize b = n + 1;
192             while (b <= node_count && rep[b]) ++b;
193             if (b > node_count) ++node_count;
194 
195             label[b] = 0;
196             state[b] = 0;
197             match[b] = match[base];
198             blossom[b].clear();
199             blossom[b].push_back(base);
200             for (usize x = u, y; x != base; x = rep[par[y]]) {
201                 blossom[b].push_back(x);
202                 y = rep[match[x]];
203                 blossom[b].push_back(y);
204                 queue_push(queue_push, y);
205             }
206             std::reverse(blossom[b].begin() + 1, blossom[b].end());
207             for (usize x = v, y; x != base; x = rep[par[y]]) {
208                 blossom[b].push_back(x);
209                 y = rep[match[x]];
210                 blossom[b].push_back(y);
211                 queue_push(queue_push, y);
212             }
213             set_representative(set_representative, b, b);
214 
215             for (usize x = 1; x <= node_count; ++x) {
216                 g[b][x] = {b, x, 0};
217                 g[x][b] = {x, b, 0};
218             }
219             std::fill(from[b].begin(), from[b].end(), 0);
220             for (usize x: blossom[b]) {
221                 for (usize y = 1; y <= node_count; ++y) {
222                     if (!g[x][y].w) continue;
223                     if (!g[b][y].w || less(delta(g[x][y]), delta(g[b][y]))) {
224                         g[b][y] = g[x][y];
225                         g[y][b] = g[y][x];
226                     }
227                 }
228                 for (usize y = 1; y <= n; ++y)
```

```
                    if (from[x][y]) from[b][y] = x;
                }
                set_slack(b);
            };
            auto expand_blossom = [&](usize b) {
                for (usize x: blossom[b])
                    set_representative(set_representative, x, x);
                usize entry = from[b][g[b][par[b]].u];
                usize pos = rotate_blossom(b, entry);
                for (usize i = 0; i < pos; i += 2) {
                    usize x = blossom[b][i], next = blossom[b][i + 1];
                    par[x] = g[next][x].u;
                    state[x] = 1;
                    state[next] = 0;
                    slack[x] = 0;
                    set_slack(next);
                    queue_push(queue_push, next);
                }
                state[entry] = 1;
                par[entry] = par[b];
                for (usize i = pos + 1; i < blossom[b].size(); ++i) {
                    usize x = blossom[b][i];
                    state[x] = -1;
                    set_slack(x);
                }
                rep[b] = 0;
            };
            auto found_edge = [&](const Edge& edge) {
                usize u = rep[edge.u], v = rep[edge.v];
                if (state[v] == -1) {
                    par[v] = edge.u;
                    state[v] = 1;
                    usize next = rep[match[v]];
                    slack[v] = slack[next] = 0;
                    state[next] = 0;
                    queue_push(queue_push, next);
                } else if (state[v] == 0) {
                    usize base = lca(u, v);
                    if (!base) {
                        augment(u, v);
                        augment(v, u);
                        return true;
```

```
271                 }
272                 add_blossom(u, base, v);
273             }
274             return false;
275         };
276         auto matching_step = [&]() {
277             std::fill(state.begin(), state.end(), -1);
278             std::fill(slack.begin(), slack.end(), 0);
279             queue.clear();
280             queue_head = 0;
281             for (usize x = 1; x <= node_count; ++x) {
282                 if (rep[x] == x && !match[x]) {
283                     par[x] = 0;
284                     state[x] = 0;
285                     queue_push(queue_push, x);
286                 }
287             }
288             if (queue.empty()) return false;
289 
290             while (true) {
291                 while (queue_head < queue.size()) {
292                     usize u = queue[queue_head++];
293                     if (state[rep[u]] == 1) continue;
294                     for (usize v = 1; v <= n; ++v) {
295                         if (!g[u][v].w || rep[u] == rep[v]) continue;
296                         if (zero(delta(g[u][v]))) {
297                             if (found_edge(g[u][v])) return true;
298                         } else {
299                             update_slack(u, rep[v]);
300                         }
301                     }
302                 }
303 
304                 Calc d = std::numeric_limits<Calc>::max();
305                 for (usize u = 1; u <= n; ++u)
306                     if (state[rep[u]] == 0) d = std::min(d, label[u]);
307                 for (usize b = n + 1; b <= node_count; ++b)
308                     if (rep[b] == b && state[b] == 1)
309                         d = std::min(d, label[b] / 2);
310                 for (usize x = 1; x <= node_count; ++x) {
311                     if (rep[x] != x || !slack[x]) continue;
312                     Calc candidate = delta(g[slack[x]][x]);
```

```
313                     if (state[x] == 0) candidate /= 2;
314                     if (state[x] != 1 && less(candidate, d)) d = candidate;
315                 }
316                 if (d == std::numeric_limits<Calc>::max()) return false;
317                 normalize(d);
318 
319                 for (usize u = 1; u <= n; ++u) {
320                     if (state[rep[u]] == 0) {
321                         if (zero(label[u] - d)) return false;
322                         label[u] -= d;
323                     } else if (state[rep[u]] == 1) {
324                         label[u] += d;
325                     }
326                     normalize(label[u]);
327                 }
328                 for (usize b = n + 1; b <= node_count; ++b) {
329                     if (rep[b] != b) continue;
330                     if (state[b] == 0) label[b] += d * 2;
331                     else if (state[b] == 1) label[b] -= d * 2;
332                     normalize(label[b]);
333                 }
334 
335                 queue.clear();
336                 queue_head = 0;
337                 for (usize x = 1; x <= node_count; ++x) {
338                     if (rep[x] == x &&
339                         slack[x] &&
340                         rep[slack[x]] != x &&
341                         zero(delta(g[slack[x]][x]))) {
342                         if (found_edge(g[slack[x]][x])) return true;
343                     }
344                 }
345                 for (usize b = n + 1; b <= node_count; ++b)
346                     if (rep[b] == b && state[b] == 1 && zero(label[b]))
347                         expand_blossom(b);
348             }
349         };
350 
351         while (matching_step());
352         std::vector<usize> result_match(n, npos);
353         for (usize u = 1; u <= n; ++u) {
354             if (match[u] && match[u] <= n) result_match[u - 1] = match[u] - 1;
```

```
355             }
356             T result{};
357             for (usize u = 0; u < n; ++u) {
358                 usize v = result_match[u];
359                 if (v == npos || result_match[v] != u || W[u][v] == no_edge)
360                     return std::nullopt;
361                 if (u < v) result += W[u][v];
362             }
363             return std::pair{result, std::move(result_match)};
364         }
365     };
366
367     }  // namespace acm
```

第 5 章 网络流

5.1 最大流

在整数容量网络中求 s 到 t 的最大流。边以链式前向星成对存储，第 i 条边的残量弧为 $2i$ 与 $2i+1$，因此可以按编号回读任意边的容量与当前流量。

5.1.1 接口

接口	说明
`MaxFlow<FlowT = i32>`	模板参数须为整型（`static_assert` 检查）；容量与流值都用 `FlowT` 表示
`MaxFlow(usize n)`	构造 n 个点、零条边的网络，编号 $[0, n)$
`usize size_V()`	点数
`usize size_E()`	边数（成对存储，故为残量弧数的一半）
`usize add_edge(u, v, cap)`	加入 $u \to v$、容量为 `cap` 的有向边，返回边编号，供 `edge()` 回读
`Edge edge(usize i)`	返回第 `i` 条边的 `{from, to, capacity, flow}`；反映调用时的残量状态
`FlowT max_flow(usize s, usize t)`	在残量网络上求最大流并返回流量；`s == t` 时返回 0

5.1.2 约定

`edge(i).capacity` 由正向与反向残量相加还原，因此 `FlowT` 必须能容纳「容量 + 已流流量」，否则溢出；无向边要正反各加一条，只加一条表示单向容量。`max_flow` 就地修改残量网络，重复调用是增量增广而非从零重算，需要重算时请重新构图。顶点数在构造时固定，没有动态加点接口；默认构造得到空图，实际应使用 `MaxFlow(n)`。

5.1.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <limits>
#include <type_traits>
#include <vector>
```

```
7
8  #include "def.hpp"
9
10 namespace acm
11 {
12
13 template <typename FlowT = i32>
14 class MaxFlow {
15     static_assert(std::is_integral_v<FlowT>);
16
17 public:
18     struct Edge {
19         usize from, to;
20         FlowT capacity, flow;
21     };
22
23 private:
24     struct Arc {
25         usize to, nxt;
26         FlowT w;
27     };
28
29     static constexpr usize npos = usize(-1);
30
31     std::vector<usize> head;
32     std::vector<Arc> e;
33
34 public:
35     MaxFlow() = default;
36     explicit MaxFlow(usize n): head(n, npos) {}
37
38     usize size_V() const { return head.size(); }
39     usize size_E() const { return e.size() / 2; }
40
41     Edge edge(usize i) const {
42         const auto &a = e[2 * i], &b = e[2 * i + 1];
43         return {b.to, a.to, a.w + b.w, b.w};
44     }
45
46     usize add_edge(usize u, usize v, FlowT cap) {
47         usize i = size_E();
48         e.push_back({v, head[u], cap});
```

```
49          head[u] = 2 * i;
50          e.push_back({u, head[v], 0});
51          head[v] = 2 * i + 1;
52          return i;
53      }
54
55      FlowT max_flow(usize s, usize t) {
56          if (s == t) return 0;
57          constexpr FlowT INF = std::numeric_limits<FlowT>::max();
58          usize n = size_V();
59          std::vector<usize> dep(n), cur(n), q(n);
60
61          auto bfs = [&] {
62              dep.assign(n, npos);
63              cur = head;
64              usize h = 0, r = 0;
65              dep[s] = 0;
66              q[r++] = s;
67              while (h < r) {
68                  usize u = q[h++];
69                  for (usize i = head[u]; i != npos; i = e[i].nxt) {
70                      if (!e[i].w || dep[e[i].to] != npos) continue;
71                      dep[e[i].to] = dep[u] + 1;
72                      q[r++] = e[i].to;
73                  }
74              }
75              return dep[t] != npos;
76          };
77          auto dfs = [&](auto&& self, usize u, FlowT flow) -> FlowT {
78              if (u == t || !flow) return flow;
79              FlowT res = flow;
80              for (usize& i = cur[u]; i != npos; i = e[i].nxt) {
81                  if (!e[i].w || dep[e[i].to] != dep[u] + 1) continue;
82                  FlowT c = self(self, e[i].to, std::min(res, e[i].w));
83                  e[i].w -= c;
84                  e[i ^ 1].w += c;
85                  if (!(res -= c)) break;
86              }
87              if (res == flow) dep[u] = npos;
88              return flow - res;
89          };
90
```

```
91          FlowT ans = 0;
92          while (bfs()) ans += dfs(dfs, s, INF);
93          return ans;
94      }
95  };
96
97  }  // namespace acm
```

5.2 最小费用流

整数容量、有符号费用的费用流，既支持最小费用最大流，也支持只沿负费用增广路推流。

5.2.1 接口

接口	说明
`MinCostFlow<FlowT = i32, CostT = i64>`	FlowT 须整型，CostT 须有符号整型
`MinCostFlow(usize n)`	建 n 个点、零条边的网络
`usize size_V() / size_E()`	点数 / 边数
`usize add_edge(u, v, cap, cost)`	加 $u \to v$、容量 cap、单位费用 cost 的边，返回边编号
`Edge edge(usize i)`	返回 {from, to, capacity, flow, cost}；反映调用时的残量状态
`min_cost_flow(s, t)`	只沿负费用增广路推流，返回 {flow, cost}
`min_cost_max_flow(s, t)`	在最大流前提下最小化费用，返回 {flow, cost}

5.2.2 约定

`min_cost_flow` 的语义是「在费用最小的前提下尽量少流」：当前最短增广路费用非负时立即停止，费用全非负时返回 {0, 0}，求最大流请用 `min_cost_max_flow`。`s == t` 时返回 {0, 0}；与最大流一样，重复调用会在残量网络上继续推进。CostT 必须有符号，费用的绝对值与总流量都要能被容纳，实现用 `numeric_limits<CostT>::max()/4` 作无穷大。

5.2.3 完整源码

```
1  #pragma once
2
3  #include <algorithm>
4  #include <functional>
5  #include <limits>
6  #include <queue>
7  #include <type_traits>
8  #include <utility>
```

```cpp
#include <vector>

#include "def.hpp"

namespace acm
{

template <typename FlowT = i32, typename CostT = i64>
class MinCostFlow {
    static_assert(std::is_integral_v<FlowT>);
    static_assert(std::is_integral_v<CostT> && std::is_signed_v<CostT>);

public:
    struct Edge {
        usize from, to;
        FlowT capacity, flow;
        CostT cost;
    };

private:
    struct Arc {
        usize to, nxt;
        FlowT w;
        CostT c;
    };

    static constexpr usize npos = usize(-1);

    std::vector<usize> head;
    std::vector<Arc> e;

    template <bool maximize>
    std::pair<FlowT, CostT> flow(usize s, usize t) {
        if (s == t) return {0, 0};
        constexpr FlowT flow_inf = std::numeric_limits<FlowT>::max();
        constexpr CostT cost_inf = std::numeric_limits<CostT>::max() / 4;
        usize n = size_V();
        std::vector<usize> dep(n), cur(n), q;
        std::vector<u8> vis(n);
        std::vector<CostT> dis(n), h(n, 0);
        q.reserve(n);

```

```
51          auto spfa = [&] {
52              dis.assign(n, cost_inf);
53              vis.assign(n, false);
54              q.clear();
55              dis[s] = 0;
56              vis[s] = true;
57              q.push_back(s);
58              for (usize hd = 0; hd < q.size(); hd++) {
59                  usize u = q[hd];
60                  vis[u] = false;
61                  for (usize i = head[u]; i != npos; i = e[i].nxt) {
62                      usize v = e[i].to;
63                      if (!e[i].w) continue;
64                      CostT tmp = dis[u] + e[i].c + h[u] - h[v];
65                      if (tmp < dis[v]) {
66                          dis[v] = tmp;
67                          if (!vis[v]) vis[v] = true, q.push_back(v);
68                      }
69                  }
70              }
71              return dis[t] != cost_inf;
72          };
73          auto dijkstra = [&] {
74              using Item = std::pair<CostT, usize>;
75              std::priority_queue<Item, std::vector<Item>, std::greater<>> q;
76              dis.assign(n, cost_inf);
77              vis.assign(n, false);
78              q.push({dis[s] = 0, s});
79              while (!q.empty()) {
80                  auto [d, u] = q.top();
81                  q.pop();
82                  if (vis[u]) continue;
83                  vis[u] = true;
84                  for (usize i = head[u]; i != npos; i = e[i].nxt) {
85                      if (!e[i].w) continue;
86                      usize v = e[i].to;
87                      CostT tmp = d + e[i].c + h[u] - h[v];
88                      if (tmp < dis[v]) q.push({dis[v] = tmp, v});
89                  }
90              }
91              return dis[t] != cost_inf;
92          };
```

```
93          auto bfs = [&] {
94              dep.assign(n, npos);
95              cur = head;
96              q.clear();
97              dep[s] = 0;
98              q.push_back(s);
99              for (usize hd = 0; hd < q.size(); hd++) {
100                 usize u = q[hd];
101                 for (usize i = head[u]; i != npos; i = e[i].nxt) {
102                     usize v = e[i].to;
103                     if (!e[i].w || dep[v] != npos || h[v] != h[u] + e[i].c)
104                         continue;
105                     dep[v] = dep[u] + 1;
106                     q.push_back(v);
107                 }
108             }
109             return dep[t] != npos;
110         };
111         auto dfs = [&](auto&& self, usize u, FlowT flow) -> FlowT {
112             if (u == t || !flow) return flow;
113             FlowT res = flow;
114             for (usize& i = cur[u]; i != npos; i = e[i].nxt) {
115                 if (!e[i].w ||
116                     dep[e[i].to] != dep[u] + 1 ||
117                     h[e[i].to] != h[u] + e[i].c)
118                     continue;
119                 FlowT c = self(self, e[i].to, std::min(res, e[i].w));
120                 e[i].w -= c;
121                 e[i ^ 1].w += c;
122                 if (!(res -= c)) break;
123             }
124             if (res == flow) dep[u] = npos;
125             return flow - res;
126         };
127
128         bool neg = std::any_of(e.begin(), e.end(), [](auto& x) {
129             return x.w > 0 && x.c < 0;
130         });
131         FlowT total_flow = 0;
132         CostT total_cost = 0;
133         while (neg ? spfa() : dijkstra()) {
134             neg = false;
```

```
135             for (usize u = 0; u < n; u++) {
136                 if (dis[u] != cost_inf) h[u] += dis[u];
137             }
138             if (!maximize && h[t] >= 0) break;
139             FlowT delta = 0;
140             while (bfs()) delta += dfs(dfs, s, flow_inf);
141             total_flow += delta;
142             total_cost += CostT(delta) * h[t];
143         }
144         return {total_flow, total_cost};
145     }
146
147 public:
148     MinCostFlow() = default;
149     explicit MinCostFlow(usize n): head(n, npos) {}
150
151     usize size_V() const { return head.size(); }
152     usize size_E() const { return e.size() / 2; }
153
154     Edge edge(usize i) const {
155         const auto& a = e[2 * i];
156         const auto& b = e[2 * i + 1];
157         return {b.to, a.to, a.w + b.w, b.w, a.c};
158     }
159
160     usize add_edge(usize u, usize v, FlowT cap, CostT cost) {
161         usize i = size_E();
162         e.push_back({v, head[u], cap, cost});
163         head[u] = 2 * i;
164         e.push_back({u, head[v], 0, -cost});
165         head[v] = 2 * i + 1;
166         return i;
167     }
168
169     std::pair<FlowT, CostT> min_cost_flow(usize s, usize t) {
170         return flow<false>(s, t);
171     }
172     std::pair<FlowT, CostT> min_cost_max_flow(usize s, usize t) {
173         return flow<true>(s, t);
174     }
175 };
176
```

```
177 } // namespace acm
```

5.3 最小费用可行环流

给定每个点的净供给与带容量、费用的有向边，求满足供给约束的最小费用可行流。

5.3.1 接口

接口	说明
`MinCostCirculation<FlowT, CostT>`	`FlowT` 与 `CostT` 都必须是有符号类型（`static_assert`）
`MinCostCirculation(usize V)`	建 V 个点、零条边、供给全为 0 的图
`FlowT& supply(usize i)`	点的净供给，可读可写；正值表示流出，总和必须为 0
`add_edge(u, v, cap, cost)`	加 $u \to v$、容量 `cap`、单位费用 `cost` 的边，返回边编号
`const Node& node(usize i)`	内部结点信息 `{supply, potential}`
`const Edge& edge(usize i)`	边信息 `{from, to, capacity, flow, cost}`；求解后 `flow` 即答案流量
`size_V() / size_E()`	点数 / 边数
`std::optional<CostT> circulation()`	返回最小费用；供给不平衡或不可行时返回 `std::nullopt`

5.3.2 约定

所有 `supply` 之和必须为 0，否则在算法开始前就返回 `nullopt`；正值表示流出，负值表示吸收。`FlowT` 与 `CostT` 都必须是有符号类型。求解会在内部临时加点、加边（人工弧），结束后恢复，`size_V()` 与 `size_E()` 与调用前一致，`edge(i).flow` 保留答案流量。不可行时返回 `nullopt`；有源汇的问题可把源汇并入供给，或直接用 `min_cost_flow.hpp`。

5.3.3 完整源码

```
1  #pragma once
2
3  #include <algorithm>
4  #include <cmath>
5  #include <numeric>
6  #include <optional>
7  #include <random>
8  #include <type_traits>
9  #include <vector>
10
11 #include "def.hpp"
12
```

```cpp
namespace acm
{

namespace detail
{

// Doubly linked lists to maintain the tree in Network Simplex.
class LinkedLists {
private:
    std::vector<usize> _pre, _nxt;

public:
    LinkedLists(usize n): _pre(n), _nxt(n) {
        std::iota(_pre.begin(), _pre.end(), 0);
        std::iota(_nxt.begin(), _nxt.end(), 0);
    }
    usize prev(usize i) { return _pre[i]; }
    usize next(usize i) { return _nxt[i]; }
    void link(usize u, usize v) { _nxt[u] = v, _pre[v] = u; }
    void erase(usize u) {
        _pre[_nxt[u]] = _pre[u];
        _nxt[_pre[u]] = _nxt[u];
        _pre[u] = _nxt[u] = u;
    }
    void insert(usize pos, usize u) { link(_pre[pos], u), link(u, pos); }
};

}  // namespace detail

template <typename FlowT, typename CostT>
class MinCostCirculation {
    static_assert(std::is_signed_v<FlowT>, "FlowT must be signed type");
    static_assert(std::is_signed_v<CostT>, "CostT must be signed type");

public:
    struct Node {
        FlowT supply;
        CostT potential;
    };
    struct Edge {
        usize from, to;
        FlowT capacity, flow;
```

```
55          CostT cost;
56      };
57
58  private:
59      enum class ArcState { UPPER = -1, LOWER = 1, TREE = 0 };
60      struct _Node: Node {
61          usize par, pre;
62      };
63      struct _Edge: Edge {
64          ArcState sta;
65      };
66
67      std::vector<_Node> _nd;
68      std::vector<_Edge> _ed;
69
70  public:
71      MinCostCirculation() = default;
72      MinCostCirculation(usize V): _nd(V) {}
73
74      usize size_V() const { return _nd.size(); }
75      usize size_E() const { return _ed.size(); }
76
77      FlowT& supply(usize i) { return _nd[i].supply; }
78      FlowT supply(usize i) const { return _nd[i].supply; }
79      const Node& node(usize i) const { return static_cast<const Node&>(_nd[i]); }
80      const Edge& edge(usize i) const { return static_cast<const Edge&>(_ed[i]); }
81      usize add_edge(usize u, usize v, FlowT cap, CostT cost) {
82          _ed.push_back(_Edge{{u, v, cap, 0, cost}, ArcState::LOWER});
83          return _ed.size() - 1;
84      }
85
86      std::optional<CostT> circulation();
87  };
88
89  template <typename FlowT, typename CostT>
90  std::optional<CostT> MinCostCirculation<FlowT, CostT>::circulation() {
91      constexpr usize npos = -1;
92
93      usize n = size_V(), m = size_E(), nxt_arc = 0, B = 0, cur_tim = 1;
94      std::vector<usize> p{}, q(n + 1), vis(n + 1);
95      detail::LinkedLists tree(2 * n + 2);
96
```

```
97      auto reduced = [&](usize e) {
98          auto u = _ed[e].from, v = _ed[e].to;
99          return _ed[e].cost + _nd[u].potential - _nd[v].potential;
100     };
101     auto sreduced = [&](usize e) {
102         return static_cast<int>(_ed[e].sta) * reduced(e);
103     };
104     auto select = [&] {
105         auto res = npos, cnt1 = B, cnt2 = m;
106         CostT mn = 0;
107         for (auto& e = nxt_arc; cnt2--; e++, (e == m) && (e = 0)) {
108             auto c = sreduced(p[e]);
109             if (c < mn) mn = c, res = p[e];
110             if (!--cnt1 && mn < 0) break;
111             else if (cnt1 == 0) cnt1 = B;
112         }
113         return res;
114     };
115     auto pivot = [&](usize in_arc) {
116         auto u_in = _ed[in_arc].from, v_in = _ed[in_arc].to;
117         auto w = [&] {
118             cur_tim++;
119             for (auto x = u_in; x != npos; x = _nd[x].par) vis[x] = cur_tim;
120             for (auto x = v_in;; x = _nd[x].par)
121                 if (vis[x] == cur_tim) return x;
122         }();
123         auto S = u_in, T = v_in, u_out = npos;
124         if (_ed[in_arc].sta == ArcState::UPPER) std::swap(S, T);
125         auto df = _ed[in_arc].capacity;
126         enum { S_SIDE, T_SIDE, SAME } side = SAME;
127         for (auto u = S; u != w && df; u = _nd[u].par) {
128             auto e = _nd[u].pre;
129             auto d =
130                 u == _ed[e].to ? _ed[e].capacity - _ed[e].flow : _ed[e].flow;
131             if (d < df) df = d, u_out = u, side = S_SIDE;
132         }
133         for (auto u = T; u != w && (df || side != T_SIDE); u = _nd[u].par) {
134             auto e = _nd[u].pre;
135             auto d =
136                 u == _ed[e].to ? _ed[e].flow : _ed[e].capacity - _ed[e].flow;
137             if (d <= df) df = d, u_out = u, side = T_SIDE;
138         }
```

```
139             if (df) {
140                 for (auto u = S; u != w; u = _nd[u].par) {
141                     auto e = _nd[u].pre;
142                     _ed[e].flow += u == _ed[e].to ? df : -df;
143                 }
144                 for (auto u = T; u != w; u = _nd[u].par) {
145                     auto e = _nd[u].pre;
146                     _ed[e].flow += u == _ed[e].to ? -df : df;
147                 }
148                 _ed[in_arc].flow += static_cast<int>(_ed[in_arc].sta) * df;
149             }
150             if (side == SAME) {
151                 auto state = static_cast<int>(_ed[in_arc].sta);
152                 _ed[in_arc].sta = static_cast<ArcState>(-state);
153                 return;
154             }
155             auto out_arc = _nd[u_out].pre;
156             _ed[in_arc].sta = ArcState::TREE;
157             _ed[out_arc].sta =
158                 _ed[out_arc].flow ? ArcState::UPPER : ArcState::LOWER;
159             u_in = S, v_in = T;
160             if (side != S_SIDE) std::swap(u_in, v_in);
161             usize ql = 0, qr = 0;
162             for (auto u = u_in; u != u_out; u = _nd[u].par) q[qr++] = u;
163             for (usize i = qr; i > 0; i--) {
164                 auto u = q[i - 1], parent = _nd[u].par;
165                 tree.erase(parent);
166                 tree.insert(u + n, parent);
167                 _nd[parent].par = u;
168                 _nd[parent].pre = _nd[u].pre;
169             }
170             tree.erase(u_in);
171             tree.insert(v_in + n, u_in);
172             _nd[u_in].par = v_in;
173             _nd[u_in].pre = in_arc;
174             auto dp = reduced(in_arc);
175             if (u_in == _ed[in_arc].from) dp = -dp;
176             for (q[ql = 0] = u_in, qr = 1; ql < qr;) {
177                 auto u = q[ql++];
178                 for (auto v = tree.next(u + n); v != u + n; v = tree.next(v))
179                     q[qr++] = v;
180                 _nd[u].potential += dp;
```

```
181             }
182         };
183         auto simplex = [&] {
184             std::mt19937 rng(std::random_device{}());
185             p.resize(_ed.size());
186             std::iota(p.begin(), p.end(), 0);
187             std::shuffle(p.begin(), p.end(), rng);
188             B = std::max(
189                 static_cast<usize>(std::ceil(std::sqrt(_ed.size()))),
190                 std::min<usize>(5, _nd.size())
191             );
192             for (usize e; (e = select()) != npos;) pivot(e);
193         };
194 
195         FlowT supply_sum = 0;
196         for (usize u = 0; u < n; u++) supply_sum += _nd[u].supply;
197         if (supply_sum != 0) return std::nullopt;
198         CostT artificial_cost = 1;
199         for (auto& e: _ed) artificial_cost += std::abs(e.cost);
200         _ed.resize(m + n);
201         _nd.push_back(_Node{{0, 0}, npos, npos});
202         for (usize e = 0; e < m; e++) _ed[e].sta = ArcState::LOWER;
203         for (usize u = 0, e = m; u < n; u++, e++) {
204             _nd[u].par = n;
205             _nd[u].pre = e;
206             tree.insert(2 * n + 1, u);
207             bool positive = _nd[u].supply >= 0;
208             if (positive) {
209                 _ed[e] =
210                     _Edge{{u, n, _nd[u].supply, _nd[u].supply, artificial_cost},
211                           ArcState::TREE};
212             } else {
213                 _ed[e] =
214                     _Edge{{n, u, -_nd[u].supply, -_nd[u].supply, artificial_cost},
215                           ArcState::TREE};
216             }
217             _nd[u].potential = positive ? -artificial_cost : artificial_cost;
218         }
219         m += n, n++, simplex(), n--, m -= n;
220         _nd.resize(n);
221         for (usize e = m; e < n + m; e++) {
222             if (_ed[e].flow != 0) {
```

```
223                 _ed.resize(m);
224                 return std::nullopt;
225             }
226         }
227         _ed.resize(m);
228         CostT cost = 0;
229         for (auto& e: _ed) cost += e.cost * e.flow;
230         return cost;
231     }
232
233 }  // namespace acm
```

第 6 章 多项式

6.1 多项式运算

以 SModint<P> 为系数的多项式，支持加减乘、带余除法，以及求逆、对数、指数、平方根、求导与积分。

6.1.1 接口

接口	说明
FPoly<P>	公有继承 std::vector<SModint<P>>；static_assert(P >= 3 && is_prime(P))
FPoly{...} / FPoly(v)	由初始化列表或 vector 构造；空多项式合法
void trim()	删除尾部零系数；大多数运算不会自动调用它
+ - * /	按多项式语义运算；另有标量乘；operator/ 是带余除法的商
== / !=	逐系数比较，不忽略尾部零，比较前建议先 trim()
static convolution(a, b)	卷积；当 $\min(\|a\|,\|b\|) \le 32$ 时走暴力
static polyinv(f)	返回 $f^{-1} \bmod x^{\|f\|}$，要求 $f_0 \neq 0$
static polyder(f) / polyint(f)	求导 / 积分；积分要求 $\|f\| < P$
static polyln(f)	返回 $\ln f \bmod x^{\|f\|}$，要求 $f_0 = 1$
static polyexp(f)	返回 $e^f \bmod x^{\|f\|}$，要求 $f_0 = 0$
static polysqrt(f)	返回 $\sqrt{f} \bmod x^{\|f\|}$，要求 $f_0 \neq 0$ 且为二次剩余
acm::inv / ln / exp / sqrt(f)	便捷包装，返回 FPoly
acm::div(f, g)	返回 {q, r}（商式与余式）；要求 g 非空且首项非零
derivative(f) / integrate(f)	求导 / 积分的自由函数包装

6.1.2 约定

模数必须是 ≥ 3 的奇素数，否则 static_assert 直接报错，$P = 2$ 不可用。NTT 长度上限是 $2^{\mathrm{ctz}(P-1)}$（对 998244353 为 2^{23}），卷积超过上限会在断言处失败。常数项要求是硬性的：求逆 $f_0 \neq 0$、对数 $f_0 = 1$、指数 $f_0 = 0$、平方根 $f_0 \neq 0$，自由函数版本的 sqrt 还要求前导零个数为偶数。FPoly 继承自 vector，尾部零会参与 size() 与比较，需要标准形式时先调用 trim()；polyint 要求 $n < P$。

6.1.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <array>
#include <bit>
#include <cassert>
#include <initializer_list>
#include <utility>
#include <vector>

#include "modint.hpp"

namespace acm
{

template <i32 P>
struct FPoly;

template <i32 P>
std::pair<FPoly<P>, FPoly<P>> div(const FPoly<P>&, const FPoly<P>&);

template <i32 P>
struct FPoly: std::vector<SModint<P>> {
    using Mint = SModint<P>;
    using Base = std::vector<Mint>;
    using Base::Base;

    static constexpr bool is_prime_const(i32 x) {
        if (x < 2) return false;
        for (i32 i = 2; i64(i) * i <= x; i++)
            if (x % i == 0) return false;
        return true;
    }

    static_assert(P >= 3 && is_prime_const(P), "P must be an odd prime");
    static constexpr int LG = __builtin_ctz(P - 1);

    static constexpr Mint primitive_root_const() {
        i32 phi = P - 1, x = phi;
        std::array<i32, 32> fac{};
        int cnt = 0;
```

```
42          for (i32 i = 2; i64(i) * i <= x; i++) {
43              if (x % i) continue;
44              fac[cnt++] = i;
45              while (x % i == 0) x /= i;
46          }
47          if (x > 1) fac[cnt++] = x;
48          for (i32 g = 2;; g++) {
49              bool ok = true;
50              for (int i = 0; i < cnt; i++)
51                  if (pow(Mint(g), phi / fac[i]) == Mint(1)) {
52                      ok = false;
53                      break;
54                  }
55              if (ok) return Mint(g);
56          }
57          return Mint(0);
58      }
59
60      static constexpr Mint G = primitive_root_const();
61      inline static std::vector<Mint> w{0, 1};
62
63      static void prepare_roots(usize n) {
64          assert(std::has_single_bit(n));
65          if (w.size() >= n) return;
66          usize h = w.size() >> 1;
67          w.resize(n);
68          for (; h * 2 <= n; h <<= 1) {
69              Mint wh = pow(G, (P - 1) / (h * 2));
70              w[h] = Mint(1);
71              for (usize j = 1; j < h; j++) w[h + j] = w[h + j - 1] * wh;
72          }
73      }
74
75      static void dft(Mint* a, usize n) {
76          if (n <= 1) return;
77          prepare_roots(n);
78          for (usize i = n >> 1; i >= 1; i >>= 1) {
79              for (usize j = 0; j < n; j += (i << 1)) {
80                  for (usize k = 0; k < i; k++) {
81                      Mint u = a[j + k], v = a[i + j + k];
82                      a[j + k] = u + v;
83                      a[i + j + k] = (u - v) * w[i + k];
```

```
                }
            }
        }
    }

    static void idft(Mint* a, usize n) {
        if (n <= 1) return;
        prepare_roots(n);
        for (usize i = 1; i < n; i <<= 1) {
            for (usize j = 0; j < n; j += (i << 1)) {
                for (usize k = 0; k < i; k++) {
                    Mint u = a[j + k], v = a[i + j + k] * w[i + k];
                    a[j + k] = u + v;
                    a[i + j + k] = u - v;
                }
            }
        }
        std::reverse(a + 1, a + n);
        Mint z = Mint(n).inv();
        for (usize i = 0; i < n; i++) a[i] *= z;
    }

    static std::vector<Mint> convolution(
        const std::vector<Mint>& a, const std::vector<Mint>& b
    ) {
        usize n = a.size(), m = b.size();
        if (!n || !m) return {};
        usize need = n + m - 1;
        if (std::min(n, m) <= 32) {
            std::vector<Mint> c(need);
            for (usize i = 0; i < n; i++)
                for (usize j = 0; j < m; j++) c[i + j] += a[i] * b[j];
            return c;
        }
        usize len = std::bit_ceil(need);
        assert(len <= (usize(1) << LG));
        std::vector<Mint> x(a), y(b);
        x.resize(len);
        y.resize(len);
        dft(x.data(), len);
        dft(y.data(), len);
        for (usize i = 0; i < len; i++) x[i] *= y[i];
```

```
126             idft(x.data(), len);
127             x.resize(need);
128             return x;
129         }
130
131         static std::vector<Mint> polyinv(const std::vector<Mint>& f) {
132             usize n = f.size();
133             assert(n && f[0] != Mint(0));
134             usize len = std::bit_ceil(n);
135             std::vector<Mint> out(len), t1(len), t2(len);
136             out[0] = f[0].inv();
137             for (usize k = 1, k2 = 2; k < len; k = k2, k2 <<= 1) {
138                 usize c = std::min(k2, n);
139                 std::copy(f.begin(), f.begin() + c, t1.begin());
140                 std::fill(t1.begin() + c, t1.begin() + k2, Mint(0));
141                 std::copy(out.begin(), out.begin() + k, t2.begin());
142                 std::fill(t2.begin() + k, t2.begin() + k2, Mint(0));
143                 dft(t1.data(), k2);
144                 dft(t2.data(), k2);
145                 for (usize i = 0; i < k2; i++) t1[i] *= t2[i];
146                 idft(t1.data(), k2);
147                 std::fill(t1.begin(), t1.begin() + k, Mint(0));
148                 dft(t1.data(), k2);
149                 for (usize i = 0; i < k2; i++) t1[i] *= t2[i];
150                 idft(t1.data(), k2);
151                 for (usize i = 0; i < k; i++) out[k + i] = -t1[k + i];
152             }
153             out.resize(n);
154             return out;
155         }
156
157         static std::vector<Mint> polyder(const std::vector<Mint>& f) {
158             usize n = f.size();
159             if (!n) return {};
160             std::vector<Mint> g(n - 1);
161             for (usize i = 1; i < n; i++) g[i - 1] = f[i] * Mint(i);
162             return g;
163         }
164
165         static std::vector<Mint> polyint(const std::vector<Mint>& f) {
166             usize n = f.size();
167             assert(n < usize(P));
```

```
168         std::vector<Mint> g(n + 1);
169         static std::vector<Mint> inv{0, 1};
170         while (inv.size() <= n) {
171             usize i = inv.size();
172             inv.push_back(-Mint(P / i) * inv[P % i]);
173         }
174         for (usize i = 1; i <= n; i++) g[i] = f[i - 1] * inv[i];
175         return g;
176     }
177
178     static std::vector<Mint> polyln(const std::vector<Mint>& f) {
179         usize n = f.size();
180         if (!n) return {};
181         assert(f[0] == Mint(1));
182         auto d = polyder(f);
183         auto g = polyinv(f);
184         auto q = convolution(d, g);
185         q.resize(n);
186         auto out = polyint(q);
187         out.resize(n);
188         return out;
189     }
190
191     static std::vector<Mint> polyexp(const std::vector<Mint>& f) {
192         usize n = f.size();
193         if (!n) return {};
194         assert(f[0] == Mint(0));
195         std::vector<Mint> g{1};
196         while (g.size() < n) {
197             usize k = std::min<usize>(g.size() * 2, n);
198             std::vector<Mint> gk(g);
199             gk.resize(k);
200             auto lg = polyln(gk);
201             for (usize i = 0; i < k; i++)
202                 lg[i] = (i < n ? f[i] : Mint(0)) - lg[i];
203             lg[0] += Mint(1);
204             g = convolution(g, lg);
205             g.resize(k);
206         }
207         g.resize(n);
208         return g;
209     }
```

```
210
211     static std::vector<Mint> polysqrt(const std::vector<Mint>& f) {
212         usize n = f.size();
213         if (!n) return {};
214         assert(f[0] != Mint(0));
215         auto r = acm::sqrt(f[0]);
216         assert(r);
217         i32 rv = (*r)();
218         Mint root = rv <= P - rv ? *r : -*r;
219         Mint inv2 = Mint(2).inv();
220         std::vector<Mint> g{root};
221         while (g.size() < n) {
222             usize k = std::min<usize>(g.size() * 2, n);
223             std::vector<Mint> gk(g);
224             gk.resize(k);
225             auto gi = polyinv(gk);
226             std::vector<Mint> fk(k);
227             for (usize i = 0; i < k && i < n; i++) fk[i] = f[i];
228             auto q = convolution(fk, gi);
229             q.resize(k);
230             g.resize(k);
231             for (usize i = 0; i < k; i++) g[i] = (g[i] + q[i]) * inv2;
232         }
233         g.resize(n);
234         return g;
235     }
236
237     FPoly() = default;
238     FPoly(std::initializer_list<Mint> l): Base(l) {}
239     FPoly(const Base& b): Base(b) {}
240     FPoly(Base&& b): Base(std::move(b)) {}
241
242     void trim() {
243         while (!this->empty() && this->back() == Mint(0)) this->pop_back();
244     }
245
246     FPoly& operator+=(const FPoly& b) {
247         if (this->size() < b.size()) this->resize(b.size());
248         for (usize i = 0; i < b.size(); i++) (*this)[i] += b[i];
249         return *this;
250     }
251     FPoly& operator-=(const FPoly& b) {
```

```
252             if (this->size() < b.size()) this->resize(b.size());
253             for (usize i = 0; i < b.size(); i++) (*this)[i] -= b[i];
254             return *this;
255         }
256         FPoly& operator*=(const FPoly& b) {
257             *this = FPoly(convolution(*this, b));
258             return *this;
259         }
260         FPoly& operator*=(Mint k) {
261             for (auto& x: *this) x *= k;
262             return *this;
263         }
264         FPoly& operator/=(const FPoly& b) {
265             *this = div(*this, b).first;
266             return *this;
267         }
268 
269         friend FPoly operator-(FPoly f) {
270             for (auto& x: f) x = -x;
271             return f;
272         }
273         friend FPoly operator+(FPoly a, const FPoly& b) { return a += b; }
274         friend FPoly operator-(FPoly a, const FPoly& b) { return a -= b; }
275         friend FPoly operator*(FPoly a, const FPoly& b) { return a *= b; }
276         friend FPoly operator*(FPoly a, Mint k) { return a *= k; }
277         friend FPoly operator*(Mint k, FPoly a) { return a *= k; }
278         friend FPoly operator/(FPoly a, const FPoly& b) { return a /= b; }
279 
280         friend bool operator==(const FPoly& a, const FPoly& b) {
281             return static_cast<const Base&>(a) == static_cast<const Base&>(b);
282         }
283         friend bool operator!=(const FPoly& a, const FPoly& b) { return !(a == b); }
284     };
285 
286     template <i32 P>
287     FPoly<P> inv(const FPoly<P>& f) {
288         return FPoly<P>(FPoly<P>::polyinv(f));
289     }
290 
291     template <i32 P>
292     FPoly<P> ln(const FPoly<P>& f) {
293         if (f.empty()) return {};
```

```
294     return FPoly<P>(FPoly<P>::polyln(f));
295 }
296
297 template <i32 P>
298 FPoly<P> exp(const FPoly<P>& f) {
299     if (f.empty()) return {};
300     return FPoly<P>(FPoly<P>::polyexp(f));
301 }
302
303 template <i32 P>
304 FPoly<P> sqrt(const FPoly<P>& f) {
305     using Mint = SModint<P>;
306     usize n = f.size();
307     if (!n) return {};
308     usize k = 0;
309     while (k < n && f[k] == Mint(0)) k++;
310     if (k == n) return FPoly<P>(n);
311     assert(k % 2 == 0);
312     FPoly<P> h(f.begin() + k, f.end());
313     h.resize(n);
314     auto s = FPoly<P>::polysqrt(h);
315     s.resize(n);
316     FPoly<P> res(n);
317     for (usize i = 0; i + k / 2 < n; i++) res[i + k / 2] = s[i];
318     return res;
319 }
320
321 template <i32 P>
322 FPoly<P> derivative(FPoly<P> f) {
323     if (f.empty()) return f;
324     return FPoly<P>(FPoly<P>::polyder(f));
325 }
326
327 template <i32 P>
328 FPoly<P> integrate(FPoly<P> f) {
329     return FPoly<P>(FPoly<P>::polyint(f));
330 }
331
332 template <i32 P>
333 std::pair<FPoly<P>, FPoly<P>> div(const FPoly<P>& f, const FPoly<P>& g) {
334     assert(!g.empty());
335     if (f.size() < g.size()) return {FPoly<P>{}, f};
```

```
336         usize n = f.size() - g.size() + 1;
337         FPoly<P> rf(f.rbegin(), f.rend());
338         FPoly<P> rg(g.rbegin(), g.rend());
339         rf.resize(n);
340         rg.resize(n);
341         FPoly<P> q = rf * inv(rg);
342         q.resize(n);
343         std::reverse(q.begin(), q.end());
344         FPoly<P> r = f - q * g;
345         r.resize(g.size() - 1);
346         r.trim();
347         return {std::move(q), std::move(r)};
348     }
349
350 }  // namespace acm
```

6.2 多项式插值

给定互异的点值，求插值多项式在某点的取值，或恢复其升幂系数；不假设节点等距。

6.2.1 接口

接口	说明
`polynomial_interpolation<T>(r, x0)`	在 x_0 处求插值多项式的值并返回 `T`
`polynomial_coefficients<T>(r, eq = F{})`	返回升幂排列的系数；`eq` 用于判断系数是否为零

6.2.2 约定

节点横坐标必须互异，实现里不检查，重复节点会导致除以零。`T` 需要支持减法、乘法与除法；浮点类型在 n 较大、节点跨度较大时数值稳定性一般。返回的系数是升幂的，长度最多为点数，尾部零会被裁掉但至少保留一个元素；空输入时求值返回 `T{0}`，系数返回长度为 1 的零多项式。

6.2.3 完整源码

```
#pragma once

#include <algorithm>
#include <functional>
#include <tuple>
#include <type_traits>
#include <utility>
```

```cpp
#include <vector>

#include "def.hpp"

namespace acm
{

namespace detail
{

template <typename T, typename R>
std::vector<std::pair<T, T>> to_pairs(R&& r) {
    std::vector<std::pair<T, T>> p;
    for (auto&& x: r) p.emplace_back(T(std::get<0>(x)), T(std::get<1>(x)));
    return p;
}

}  // namespace detail

template <typename T, typename R>
T polynomial_interpolation(const R& r, T x0) {
    auto p = detail::to_pairs<T>(r);
    usize n = p.size();
    if (!n) return T{0};
    T res{0};
    for (usize i = 0; i < n; i++) {
        T num{p[i].second}, den{1};
        for (usize j = 0; j < n; j++) {
            if (i == j) continue;
            num = num * (x0 - p[j].first);
            den = den * (p[i].first - p[j].first);
        }
        res += num / den;
    }
    return res;
}

template <typename T, typename R, typename F = std::equal_to<>>
std::vector<T> polynomial_coefficients(const R& r, F eq = F{}) {
    auto p = detail::to_pairs<T>(r);
    usize n = p.size();
    if (!n) return {T{0}};
```

```
50      std::vector<T> A(n + 1, T{0}), B(n, T{0}), res(n, T{0});
51      A[0] = T{1};
52      for (usize i = 0; i < n; i++) {
53          for (usize j = n; j; j--) A[j] = A[j - 1] - p[i].first * A[j];
54          A[0] *= (-p[i].first);
55      }
56      for (usize i = 0; i < n; i++) {
57          auto [x, y] = p[i];
58          if (eq(x, T{0})) {
59              std::copy(A.begin() + 1, A.end(), B.begin());
60          } else {
61              T tmp = -T{1} / x;
62              B[0] = A[0] * tmp;
63              for (usize j = 1; j < n; j++) B[j] = (A[j] - B[j - 1]) * tmp;
64          }
65          T tmp{1};
66          for (usize j = 0; j < n; j++)
67              if (i != j) tmp *= (x - p[j].first);
68          tmp = y / tmp;
69          for (usize j = 0; j < n; j++) res[j] += B[j] * tmp;
70      }
71      while (res.size() > 1 && eq(res.back(), T{0})) res.pop_back();
72      return res;
73  }
74
75  }  // namespace acm
```

第 7 章 计算几何

7.1 计算几何基础

二维向量与点、由「点 + 方向」表示的直线与线段，以及方向判定、投影、交点等基础谓词。

7.1.1 接口

接口	说明
`Vec2<T> / Point2<T>`	坐标 x、y；Point2 只是同类型别名
`cast<U>()`	显式转换坐标类型，不做隐式转换
`+ - * /`	向量加减、标量乘除与复合赋值；除法只在浮点 T 上可用
`== != <`	精确比较；浮点请改用 cmp，否则等于「逐位相同」
`cmp(a, b) / sgn(x)`	三值比较，返回 $-1/0/1$；浮点用绝对容差 eps
`dot(a, b) / cross(a, b)`	点积 / 叉积；返回 Wide<T>
`cross(o, a, b)`	以 o 为原点的叉积，即 $(a-o)\times(b-o)$
`norm_sq / distance_sq`	平方长度 / 平方距离；返回 Wide<T>
`norm / distance / angle / rotate`	模长、距离、极角、旋转；仅浮点 T
`normalized(a)`	单位向量；零向量返回 nullopt

7.1.1.1 直线、线段与交点

接口	说明
`Line2<T>{p, d}`	点 + 方向表示直线；方向必须非零
`Line2<T>::through(a, b)`	过两点的直线；要求 $b-a$ 能由 T 表示
`valid()`	方向非零时为真；多数几何函数在入口断言它
`Segment2<T>{a, b}`	线段；line() 返回所在直线，要求 $a \neq b$
`line_side(l, q)`	点在直线左侧 / 上 / 右侧，返回 $1/0/-1$
`orientation(o, a, b)`	三点转向，返回 $1/0/-1$

接口	说明
parallel / perpendicular	平行 / 垂直判定
on_line / on_segment	点是否在直线 / 线段上；后者对浮点带 eps 容差
project / reflect / distance(p, l)	投影、对称点、点到直线距离；仅浮点
LineIntersectionKind	枚举：none、point、coincident
LineIntersection2<T>{kind, point}	交点结果；coincident 时 point 无意义
intersection(Line2, Line2)	直线交点；仅浮点 T
intersection(Segment2, Segment2)	线段交点；退化为点、共线重叠都单独处理；仅浮点 T
intersects(Line2, Line2)	交点是否存在的布尔包装

7.1.2 约定

坐标类型受 GeomScalar 约束：有符号整数、浮点或 i128，不接受无符号整数与 bool。整数运算前把坐标提升到 Wide<T>：不超过 4 字节的整型提升为 i64，8 字节提升为 i128；结果仍可能溢出，这种情形不受支持。浮点比较统一使用绝对容差 eps，默认 10^{-9}，可在包含任何几何头文件之前用 ACM_EPS 覆盖，且必须对整个程序统一。零方向直线无效，多数函数在入口断言；Line2::through 会计算 $b-a$，端点差需能由 T 表示；直线交点只对浮点类型实例化，整数坐标需要先 cast<long double>()。

7.1.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <cassert>
#include <cmath>
#include <concepts>
#include <optional>
#include <type_traits>
#include <utility>

#include "def.hpp"

namespace acm
{

#ifdef ACM_EPS
inline constexpr long double eps = ACM_EPS;
#else
inline constexpr long double eps = 1e-9L;
```

```
#endif

template <typename T>
concept GeomScalar = (std::signed_integral<T> ||
                      std::floating_point<T> ||
                      std::same_as<std::remove_cv_t<T>, i128>) &&
    !std::same_as<std::remove_cv_t<T>, bool>;

template <GeomScalar T>
using Wide = std::conditional_t<
    std::is_integral_v<T>,
    std::conditional_t<(sizeof(T) <= 4), i64, i128>,
    T
>;

template <GeomScalar T>
using Real = std::conditional_t<std::is_integral_v<T>, long double, T>;

template <GeomScalar T>
constexpr int cmp(T a, T b) {
    if constexpr (std::floating_point<T>)
        if (std::abs(a - b) <= T(eps)) return 0;
    return (a > b) - (a < b);
}

template <GeomScalar T>
constexpr int sgn(T x) {
    return cmp(x, T{});
}

template <GeomScalar T>
struct Vec2 {
    T x{}, y{};

    constexpr Vec2() = default;
    constexpr Vec2(T x, T y): x(x), y(y) {}

    template <GeomScalar U>
    constexpr Vec2<U> cast() const {
        return Vec2<U>{(U)x, (U)y};
    }

```

```
62      constexpr Vec2 operator+() const { return *this; }
63      constexpr Vec2 operator-() const { return {-x, -y}; }
64      constexpr Vec2& operator+=(Vec2 a) { return x += a.x, y += a.y, *this; }
65      constexpr Vec2& operator-=(Vec2 a) { return x -= a.x, y -= a.y, *this; }
66      constexpr Vec2& operator*=(T k) { return x *= k, y *= k, *this; }
67      constexpr Vec2& operator/=(T k)
68          requires std::floating_point<T>
69      {
70          return x /= k, y /= k, *this;
71      }
72
73      friend constexpr Vec2 operator+(Vec2 a, Vec2 b) { return a += b; }
74      friend constexpr Vec2 operator-(Vec2 a, Vec2 b) { return a -= b; }
75      friend constexpr Vec2 operator*(Vec2 a, T k) { return a *= k; }
76      friend constexpr Vec2 operator*(T k, Vec2 a) { return a *= k; }
77      friend constexpr Vec2 operator/(Vec2 a, T k)
78          requires std::floating_point<T>
79      {
80          return a /= k;
81      }
82
83      friend constexpr bool operator==(Vec2 a, Vec2 b) {
84          return a.x == b.x && a.y == b.y;
85      }
86      friend constexpr bool operator!=(Vec2 a, Vec2 b) { return !(a == b); }
87      friend constexpr bool operator<(Vec2 a, Vec2 b) {
88          return a.x != b.x ? a.x < b.x : a.y < b.y;
89      }
90  };
91
92  template <GeomScalar T>
93  using Point2 = Vec2<T>;
94
95  template <GeomScalar T>
96  constexpr int cmp(Vec2<T> a, Vec2<T> b) {
97      if (int c = cmp(a.x, b.x)) return c;
98      return cmp(a.y, b.y);
99  }
100
101 template <GeomScalar T>
102 constexpr Wide<T> dot(Vec2<T> a, Vec2<T> b) {
103     using W = Wide<T>;
```

```
104     return W(a.x) * W(b.x) + W(a.y) * W(b.y);
105 }
106
107 template <GeomScalar T>
108 constexpr Wide<T> cross(Vec2<T> a, Vec2<T> b) {
109     using W = Wide<T>;
110     return W(a.x) * W(b.y) - W(a.y) * W(b.x);
111 }
112
113 template <GeomScalar T>
114 constexpr Wide<T> cross(Vec2<T> o, Vec2<T> a, Vec2<T> b) {
115     using W = Wide<T>;
116     return (W(a.x) - W(o.x)) * (W(b.y) - W(o.y)) -
117         (W(a.y) - W(o.y)) * (W(b.x) - W(o.x));
118 }
119
120 template <GeomScalar T>
121 constexpr Wide<T> norm_sq(Vec2<T> a) {
122     return dot(a, a);
123 }
124
125 template <GeomScalar T>
126 constexpr Wide<T> distance_sq(Vec2<T> a, Vec2<T> b) {
127     return norm_sq(a - b);
128 }
129
130 template <std::floating_point T>
131 T norm(Vec2<T> a) {
132     return std::hypot(a.x, a.y);
133 }
134
135 template <std::floating_point T>
136 T distance(Vec2<T> a, Vec2<T> b) {
137     return std::hypot(a.x - b.x, a.y - b.y);
138 }
139
140 template <std::floating_point T>
141 T angle(Vec2<T> a) {
142     return std::atan2(a.y, a.x);
143 }
144
145 template <std::floating_point T>
```

```cpp
Vec2<T> rotate(Vec2<T> a, T r) {
    T c = std::cos(r), s = std::sin(r);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}

template <std::floating_point T>
std::optional<Vec2<T>> normalized(Vec2<T> a) {
    T d = norm(a);
    if (!cmp(d, T{})) return std::nullopt;
    return a / d;
}

template <GeomScalar T>
struct Line2 {
    Vec2<T> p{}, d{};

    static constexpr Line2 through(Vec2<T> a, Vec2<T> b) { return {a, b - a}; }
    constexpr bool valid() const { return d.x != T{} || d.y != T{}; }
};

template <GeomScalar T>
struct Segment2 {
    Vec2<T> a{}, b{};

    constexpr Line2<T> line() const {
        assert(a != b);
        return Line2<T>::through(a, b);
    }
};

namespace detail
{

// Scale a direction so that its larger component has magnitude one;
// a zero vector stays zero.
template <std::floating_point T>
constexpr Vec2<T> max_norm_scale(Vec2<T> v) {
    T n = std::max(std::abs(v.x), std::abs(v.y));
    return n == T{} ? Vec2<T>{} : Vec2<T>{v.x / n, v.y / n};
}

}  // namespace detail
```

```cpp

template <GeomScalar T>
constexpr int line_side(Line2<T> l, Vec2<T> q) {
    if constexpr (std::floating_point<T>)
        return sgn(cross(detail::max_norm_scale(l.d), q - l.p));
    else {
        using W = Wide<T>;
        W x = W(q.x) - W(l.p.x), y = W(q.y) - W(l.p.y);
        return sgn(W(l.d.x) * y - W(l.d.y) * x);
    }
}

template <GeomScalar T>
constexpr int orientation(Vec2<T> o, Vec2<T> a, Vec2<T> b) {
    if constexpr (std::floating_point<T>)
        return sgn(
            cross(detail::max_norm_scale(a - o), detail::max_norm_scale(b - o))
        );
    return sgn(cross(o, a, b));
}

template <GeomScalar T>
constexpr bool parallel(Line2<T> a, Line2<T> b) {
    assert(a.valid() && b.valid());
    return orientation(Vec2<T>{}, a.d, b.d) == 0;
}

template <GeomScalar T>
constexpr bool perpendicular(Line2<T> a, Line2<T> b) {
    assert(a.valid() && b.valid());
    if constexpr (std::floating_point<T>)
        return !sgn(
            dot(detail::max_norm_scale(a.d), detail::max_norm_scale(b.d))
        );
    return dot(a.d, b.d) == 0;
}

template <GeomScalar T>
constexpr bool on_line(Line2<T> l, Vec2<T> p) {
    assert(l.valid());
    return line_side(l, p) == 0;
}
```

```cpp

template <GeomScalar T>
constexpr bool on_segment(Vec2<T> a, Vec2<T> b, Vec2<T> p) {
    if (orientation(a, b, p)) return false;
    auto in = [&](T x, T l, T r) {
        if (l > r) std::swap(l, r);
        if constexpr (std::floating_point<T>)
            return l - T(eps) <= x && x <= r + T(eps);
        return l <= x && x <= r;
    };
    return in(p.x, a.x, b.x) && in(p.y, a.y, b.y);
}

template <std::floating_point T>
Vec2<T> project(Vec2<T> p, Line2<T> l) {
    assert(l.valid());
    return l.p + l.d * (dot(p - l.p, l.d) / dot(l.d, l.d));
}

template <std::floating_point T>
Vec2<T> reflect(Vec2<T> p, Line2<T> l) {
    return project(p, l) * T{2} - p;
}

template <std::floating_point T>
T distance(Vec2<T> p, Line2<T> l) {
    return distance(p, project(p, l));
}

enum class LineIntersectionKind { none, point, coincident };

template <std::floating_point T>
struct LineIntersection2 {
    LineIntersectionKind kind = LineIntersectionKind::none;
    Vec2<T> point{};
};

template <std::floating_point T>
LineIntersection2<T> intersection(Line2<T> a, Line2<T> b) {
    assert(a.valid() && b.valid());
    // Both directions are max-norm scaled, so the denominator stays O(1) for
    // any direction magnitude; the parameter is measured in units of u, which
```

```
272     // cancels the scaling factor of a.d exactly.
273     Vec2<T> u = detail::max_norm_scale(a.d), v = detail::max_norm_scale(b.d);
274     Vec2<T> o = b.p - a.p;
275     T den = cross(u, v);
276     if (!cmp(den, T{}))
277         return {
278             on_line(a, b.p) ? LineIntersectionKind::coincident
279                             : LineIntersectionKind::none,
280             {},
281         };
282     return {LineIntersectionKind::point, a.p + u * (cross(o, v) / den)};
283 }
284
285 template <std::floating_point T>
286 LineIntersection2<T> intersection(Segment2<T> a, Segment2<T> b) {
287     if (a.a == a.b)
288         return on_segment(b.a, b.b, a.a)
289             ? LineIntersection2<T>{LineIntersectionKind::point, a.a}
290             : LineIntersection2<T>{};
291     if (b.a == b.b)
292         return on_segment(a.a, a.b, b.a)
293             ? LineIntersection2<T>{LineIntersectionKind::point, b.a}
294             : LineIntersection2<T>{};
295
296     auto r = intersection(a.line(), b.line());
297     if (r.kind == LineIntersectionKind::none) return r;
298     int s1 = orientation(a.a, a.b, b.a), s2 = orientation(a.a, a.b, b.b);
299     int s3 = orientation(b.a, b.b, a.a), s4 = orientation(b.a, b.b, a.b);
300     if (r.kind == LineIntersectionKind::point)
301         return s1 * s2 <= 0 && s3 * s4 <= 0 ? r : LineIntersection2<T>{};
302
303     // Coincident lines: both segments share a direction, so their overlap is
304     // a one-dimensional interval on the dominant axis of a.
305     T dx = a.b.x - a.a.x, dy = a.b.y - a.a.y;
306     bool use_x = std::abs(dx) >= std::abs(dy);
307     auto coord = [use_x](Vec2<T> p) { return use_x ? p.x : p.y; };
308     T a1 = coord(a.a), a2 = coord(a.b), b1 = coord(b.a), b2 = coord(b.b);
309     T lo = std::max(std::min(a1, a2), std::min(b1, b2));
310     T hi = std::min(std::max(a1, a2), std::max(b1, b2));
311     if (cmp(hi, lo) < 0) return {};
312     if (cmp(hi, lo) > 0) return {LineIntersectionKind::coincident, {}};
313     for (auto p: {a.a, a.b, b.a, b.b})
```

```
314             if (cmp(coord(p), lo) == 0) return {LineIntersectionKind::point, p};
315         return {LineIntersectionKind::coincident, {}};
316     }
317
318     template <std::floating_point T>
319     bool intersects(Line2<T> a, Line2<T> b) {
320         return intersection(a, b).kind != LineIntersectionKind::none;
321     }
322
323     template <std::floating_point T>
324     bool intersects(Segment2<T> a, Segment2<T> b) {
325         return intersection(a, b).kind != LineIntersectionKind::none;
326     }
327
328     } // namespace acm
```

7.2 圆

圆的基本判定与求交，以及两个常用构造：由三点求外接圆、对点集求最小覆盖圆。

7.2.1 接口

接口	说明
`Circle2<T>{o, r}`	圆心与半径；`valid()` 要求 $r \geq 0$（零半径合法）
`on_circle(c, p)` / `contains(c, p)`	点在圆周上 / 在圆内或圆上；整数用平方比较，浮点用 `eps`
`distance_to_circle(p, c)`	点到圆周的距离；仅浮点
`distance_to_disk(p, c)`	点到圆盘的距离（内部为 0）；仅浮点
`PointIntersectionKind`	枚举：`none`、`one`、`two`、`coincident`
`PointIntersection2<T>`	结果 `{kind, points[2]}`；`count()` 返回 0/1/2
`intersection(Line2, Circle2)`	直线与圆求交；仅浮点 `T`
`intersection(Circle2, Circle2)`	两圆求交；同心同半径时返回 `coincident`
`circle_from(a, b, c)`	三点外接圆；共线时取最远点对为直径；仅浮点 `T`
`minimum_enclosing_circle(points)`	返回 `std::optional<Circle2<T>>`；空集为 `nullopt`；仅浮点 `T`

7.2.2 约定

负半径圆无效，相关函数在入口断言 `valid()`；零半径圆是合法的退化情形。`circle_from`、直线与圆求交、两圆求交以及 `minimum_enclosing_circle` 只对浮点坐标实例化，整数坐标请先 `cast<long`

double>()，只有 contains 与 on_circle 对整数类型可用。三点共线时 circle_from 返回以最远两点为直径的圆，不报错。空点集的最小覆盖圆为 nullopt，该函数内部用 std::random_device 打乱点集，结果正确但每次点序不同。

7.2.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <array>
#include <cassert>
#include <cmath>
#include <optional>
#include <random>
#include <vector>

#include "../geometry.hpp"

namespace acm
{

template <GeomScalar T>
struct Circle2 {
    Vec2<T> o{};
    T r{};

    constexpr bool valid() const { return r >= T{}; }
};

template <GeomScalar T>
constexpr bool on_circle(Circle2<T> c, Vec2<T> p) {
    assert(c.valid());
    if constexpr (std::floating_point<T>)
        return !cmp(distance(c.o, p), c.r);
    else {
        using W = Wide<T>;
        return distance_sq(c.o, p) == W(c.r) * W(c.r);
    }
}

template <GeomScalar T>
constexpr bool contains(Circle2<T> c, Vec2<T> p) {
    assert(c.valid());
```

```
    if constexpr (std::floating_point<T>)
        return cmp(distance(c.o, p), c.r) <= 0;
    else {
        using W = Wide<T>;
        return distance_sq(c.o, p) <= W(c.r) * W(c.r);
    }
}

template <std::floating_point T>
T distance_to_circle(Vec2<T> p, Circle2<T> c) {
    assert(c.valid());
    return std::abs(distance(p, c.o) - c.r);
}

template <std::floating_point T>
T distance_to_disk(Vec2<T> p, Circle2<T> c) {
    assert(c.valid());
    return std::max(T{}, distance(p, c.o) - c.r);
}

enum class PointIntersectionKind { none, one, two, coincident };

template <std::floating_point T>
struct PointIntersection2 {
    PointIntersectionKind kind = PointIntersectionKind::none;
    std::array<Vec2<T>, 2> points{};

    usize count() const {
        return kind == PointIntersectionKind::one ? 1
            : kind == PointIntersectionKind::two  ? 2
                                                  : 0;
    }
};

namespace detail
{

// Half chord length sqrt(r^2 - d^2), factored to avoid cancellation.
template <std::floating_point T>
T chord_half(T r, T d) {
    return std::sqrt(std::max(T{}, r - d)) * std::sqrt(std::max(T{}, r + d));
}
```

```cpp

} // namespace detail

template <std::floating_point T>
PointIntersection2<T> intersection(Line2<T> l, Circle2<T> c) {
    assert(l.valid() && c.valid());
    Vec2<T> p = project(c.o, l);
    T x = distance(c.o, p);
    int s = cmp(x, c.r);
    if (s > 0) return {};
    if (!s) return {PointIntersectionKind::one, {p, {}}};
    T h = detail::chord_half(c.r, x);
    Vec2<T> d = l.d / norm(l.d);
    return {PointIntersectionKind::two, {p - d * h, p + d * h}};
}

template <std::floating_point T>
PointIntersection2<T> intersection(Circle2<T> a, Circle2<T> b) {
    assert(a.valid() && b.valid());
    Vec2<T> d = b.o - a.o;
    T z = distance(a.o, b.o);
    if (!cmp(z, T{})) {
        if (cmp(a.r, b.r)) return {};
        if (!cmp(a.r, T{})) return {PointIntersectionKind::one, {a.o, {}}};
        return {PointIntersectionKind::coincident, {}};
    }
    T sum = a.r + b.r, dif = std::abs(a.r - b.r);
    if (cmp(z, sum) > 0 || cmp(z, dif) < 0) return {};
    T t = ((a.r - b.r) * ((a.r + b.r) / z) + z) / T{2};
    Vec2<T> m = a.o + d * (t / z);
    if (!cmp(z, sum) || !cmp(z, dif))
        return {PointIntersectionKind::one, {m, {}}};
    T h = detail::chord_half(a.r, t);
    Vec2<T> v{-d.y, d.x};
    v *= h / z;
    return {PointIntersectionKind::two, {m + v, m - v}};
}

namespace detail
{

template <std::floating_point T>
```

```cpp
Circle2<T> circle_from_pair(Vec2<T> a, Vec2<T> b) {
    Vec2<T> o{(a.x + b.x) / T{2}, (a.y + b.y) / T{2}};
    return {o, distance(o, a)};
}

}  // namespace detail

template <std::floating_point T>
Circle2<T> circle_from(Vec2<T> a, Vec2<T> b, Vec2<T> c) {
    Vec2<T> u = b - a, v = c - a;
    T scale =
        std::max({std::abs(u.x), std::abs(u.y), std::abs(v.x), std::abs(v.y)});
    if (scale == T{}) return detail::circle_from_pair(a, b);
    Vec2<T> su = u / scale, sv = v / scale;
    T d = T{2} * cross(su, sv);
    if (!cmp(d, T{})) {
        T ab = distance(a, b), ac = distance(a, c), bc = distance(b, c);
        if (ab >= ac && ab >= bc) return detail::circle_from_pair(a, b);
        if (ac >= bc) return detail::circle_from_pair(a, c);
        return detail::circle_from_pair(b, c);
    }
    T u2 = norm_sq(su), v2 = norm_sq(sv);
    Vec2<T> o = a +
        Vec2<T>{
            (u2 * sv.y - v2 * su.y) / d,
            (su.x * v2 - sv.x * u2) / d,
        } * scale;
    return {o, distance(o, a)};
}

template <std::floating_point T>
std::optional<Circle2<T>> minimum_enclosing_circle(std::vector<Vec2<T>> p) {
    if (p.empty()) return std::nullopt;
    std::random_device rd;
    std::mt19937_64 rng(rd());
    std::shuffle(p.begin(), p.end(), rng);

    std::optional<Circle2<T>> c;
    for (usize i = 0; i < p.size(); i++) {
        if (c && contains(*c, p[i])) continue;
        c = Circle2<T>{p[i], T{}};
        for (usize j = 0; j < i; j++) {
```

```
164                 if (contains(*c, p[j])) continue;
165                 c = detail::circle_from_pair(p[i], p[j]);
166                 for (usize k = 0; k < j; k++) {
167                     if (contains(*c, p[k])) continue;
168                     c = circle_from(p[i], p[j], p[k]);
169                 }
170             }
171         }
172         return c;
173     }
174
175 }  // namespace acm
```

7.3 多边形

逆时针存储的多边形：面积、凸性、点与多边形的位置关系，以及凸包、凸多边形 Minkowski 和与半平面交。

7.3.1 接口

接口	说明
`Polygon<T>{p}`	顶点序列，要求逆时针；构造时自动去尾重复点并规范方向
`Polygon(vector<Vec2<T>>)`	按给定顶点构造；空序列合法
`vertices() / size() / operator[]`	只读访问顶点
`edges()`	返回每条边对应的 `Segment2`
`area()`	有向面积的绝对值，整数坐标返回 `long double`
`is_convex()`	逆时针意义下是否凸；少于 3 个点时返回 true
`relation(q)`	返回 `outside` / `boundary` / `inside`
`ConvexHull: convex_hull(points)`	Graham 凸包；会丢弃共线点，返回严格凸包
`minkowski_sum(a, b)`	两个凸多边形的 Minkowski 和；要求两者都凸
`half_plane_intersection(lines)`	返回 `optional<Polygon>`；左侧为可行域，仅浮点 `T`
`PointPolygonRelation`	枚举：`outside`、`boundary`、`inside`

7.3.2 约定

顶点必须逆时针；用构造函数传入顶点时会去掉与首点重合的尾点并自动规范方向，直接写 `p` 字段时需自行保证顺序。`area()` 返回有向面积的绝对值，整数坐标返回 `long double`。`convex_hull` 丢弃共线点，返回严格凸包；`minkowski_sum` 只在调试构建中断言输入为凸，非凸输入不保证结果。`half_plane_`

`intersection` 只接受浮点坐标，并且只会在交集为有界、非退化多边形时返回结果：空集、无界区域、点或线段都返回 `nullopt`。

7.3.3 完整源码

```cpp
#pragma once

#include <algorithm>
#include <cassert>
#include <cmath>
#include <deque>
#include <optional>
#include <utility>
#include <vector>

#include "../geometry.hpp"

namespace acm
{

enum class PointPolygonRelation { outside, boundary, inside };

namespace detail
{

template <GeomScalar T>
Wide<T> area2(const std::vector<Vec2<T>>& p) {
    Wide<T> s{};
    for (usize i = 1; i + 1 < p.size(); i++) s += cross(p[0], p[i], p[i + 1]);
    return s;
}

template <GeomScalar T>
bool outside(Line2<T> l, Vec2<T> p) {
    return line_side(l, p) < 0;
}

}  // namespace detail

template <GeomScalar T>
struct Polygon {
    std::vector<Vec2<T>> p;

```

```
39      Polygon() = default;
40      explicit Polygon(std::vector<Vec2<T>> q): p(std::move(q)) {
41          if (p.size() > 1 && p.front() == p.back()) p.pop_back();
42          if (detail::area2(p) < 0) std::reverse(p.begin(), p.end());
43      }
44
45      const std::vector<Vec2<T>>& vertices() const { return p; }
46      usize size() const { return p.size(); }
47      const Vec2<T>& operator[](usize i) const { return p[i]; }
48
49      std::vector<Segment2<T>> edges() const {
50          std::vector<Segment2<T>> res;
51          res.reserve(p.size());
52          for (usize i = 0; i < p.size(); i++)
53              res.push_back({p[i], p[(i + 1) % p.size()]});
54          return res;
55      }
56
57      Real<T> area() const {
58          using R = Real<T>;
59          R s = R(detail::area2(p));
60          return (s < 0 ? -s : s) / R{2};
61      }
62
63      bool is_convex() const {
64          if (p.size() < 3) return true;
65          for (usize i = 0; i < p.size(); i++)
66              if (orientation(
67                      p[i], p[(i + 1) % p.size()], p[(i + 2) % p.size()]
68                  ) < 0)
69                  return false;
70          return true;
71      }
72
73      PointPolygonRelation relation(Vec2<T> q) const {
74          if (p.empty()) return PointPolygonRelation::outside;
75          if (p.size() == 1)
76              return sgn(norm_sq(p[0] - q)) ? PointPolygonRelation::outside
77                                            : PointPolygonRelation::boundary;
78          bool in = false;
79          for (usize i = 0; i < p.size(); i++) {
80              Vec2<T> a = p[i], b = p[(i + 1) % p.size()];
```

```
            int s = orientation(a, b, q);
            if ((a.y <= q.y && q.y < b.y && s > 0) ||
                (b.y <= q.y && q.y < a.y && s < 0))
                in = !in;
            if (on_segment(a, b, q)) return PointPolygonRelation::boundary;
        }
        return in ? PointPolygonRelation::inside
                  : PointPolygonRelation::outside;
    }
};

template <GeomScalar T>
std::vector<Vec2<T>> convex_hull(std::vector<Vec2<T>> p) {
    usize n = p.size();
    if (n <= 1) return p;

    std::swap(p[0], *std::min_element(p.begin(), p.end()));
    std::sort(p.begin() + 1, p.end(), [&](Vec2<T> a, Vec2<T> b) {
        int c = orientation(p[0], a, b);
        if (c) return c > 0;
        return norm_sq(a - p[0]) < norm_sq(b - p[0]);
    });
    std::vector<Vec2<T>> h;
    h.reserve(n);
    for (auto q: p) {
        while (h.size() >= 2 && orientation(h[h.size() - 2], h.back(), q) <= 0)
            h.pop_back();
        h.push_back(q);
    }
    return h;
}

template <GeomScalar T>
Polygon<T> minkowski_sum(const Polygon<T>& a, const Polygon<T>& b) {
    assert(a.is_convex() && b.is_convex());
    if (a.size() == 0 || b.size() == 0) return {};
    if (a.size() == 1 && b.size() == 1) return Polygon<T>{{a[0] + b[0]}};

    auto get_edges = [&](const std::vector<Vec2<T>>& p) {
        usize s = std::min_element(p.begin(), p.end()) - p.begin();
        std::vector<Vec2<T>> e;
        e.reserve(p.size());
```

```
123         for (usize i = s; i + 1 < p.size(); i++) e.push_back(p[i + 1] - p[i]);
124         e.push_back(p[0] - p.back());
125         for (usize i = 0; i < s; i++) e.push_back(p[i + 1] - p[i]);
126         return std::pair{std::move(e), p[s]};
127     };
128     auto [e1, p0] = get_edges(a.vertices());
129     auto [e2, q0] = get_edges(b.vertices());
130 
131     std::vector<Vec2<T>> res{p0 + q0};
132     for (usize i = 0, j = 0; i != e1.size() || j != e2.size();) {
133         int c = i == e1.size() ? 1
134             : j == e2.size()   ? -1
135                                : sgn(cross(e2[j], e1[i]));
136         Vec2<T> d{};
137         if (c <= 0) d += e1[i++];
138         if (c >= 0) d += e2[j++];
139         res.push_back(res.back() + d);
140     }
141     if (res.size() > 1) res.pop_back();
142     return Polygon<T>{std::move(res)};
143 }
144 
145 template <std::floating_point T>
146 std::optional<Polygon<T>> half_plane_intersection(std::vector<Line2<T>> ls) {
147     if (ls.empty()) return std::nullopt;
148 
149     struct HP {
150         Line2<T> l;
151         T a;
152     };
153     std::vector<HP> h;
154     h.reserve(ls.size());
155     T pi = std::acos(T(-1));
156     for (auto l: ls) {
157         assert(l.valid());
158         T a = angle(l.d);
159         if (a < T{}) a += T{2} * pi;
160         h.push_back({l, a});
161     }
162     std::sort(h.begin(), h.end(), [](const HP& x, const HP& y) {
163         return x.a < y.a;
164     });
```

```
165
166         std::vector<Line2<T>> u;
167         u.reserve(h.size());
168         for (auto x: h) {
169             auto l = x.l;
170             if (!u.empty() && parallel(u.back(), l) && dot(u.back().d, l.d) > T{}) {
171                 if (detail::outside(l, u.back().p)) u.back() = l;
172             } else {
173                 u.push_back(l);
174             }
175         }
176
177         auto inter = [&](Line2<T> a, Line2<T> b) -> std::optional<Vec2<T>> {
178             auto r = intersection(a, b);
179             if (r.kind != LineIntersectionKind::point) return std::nullopt;
180             return r.point;
181         };
182
183         std::deque<Line2<T>> q;
184         for (auto l: u) {
185             while (q.size() > 1) {
186                 auto x = inter(q[q.size() - 2], q.back());
187                 if (!x) return std::nullopt;
188                 if (!detail::outside(l, *x)) break;
189                 q.pop_back();
190             }
191             while (q.size() > 1) {
192                 auto x = inter(q[0], q[1]);
193                 if (!x) return std::nullopt;
194                 if (!detail::outside(l, *x)) break;
195                 q.pop_front();
196             }
197             q.push_back(l);
198         }
199         while (q.size() > 2) {
200             auto x = inter(q[q.size() - 2], q.back());
201             if (!x) return std::nullopt;
202             if (!detail::outside(q.front(), *x)) break;
203             q.pop_back();
204         }
205         while (q.size() > 2) {
206             auto x = inter(q[0], q[1]);
```

```
207             if (!x) return std::nullopt;
208             if (!detail::outside(q.back(), *x)) break;
209             q.pop_front();
210         }
211         if (q.size() < 3) return std::nullopt;
212 
213         std::vector<Vec2<T>> p;
214         p.reserve(q.size());
215         for (usize i = 0; i < q.size(); i++) {
216             if (orientation(Vec2<T>{}, q[i].d, q[(i + 1) % q.size()].d) <= 0)
217                 return std::nullopt;
218             auto x = inter(q[i], q[(i + 1) % q.size()]);
219             if (!x) return std::nullopt;
220             if (p.empty() || cmp(p.back(), *x) != 0) p.push_back(*x);
221         }
222         if (p.size() > 1 && !cmp(p.front(), p.back())) p.pop_back();
223         if (p.size() < 3) return std::nullopt;
224         return Polygon<T>(std::move(p));
225     }
226 
227 }  // namespace acm
```

第 8 章 算法

8.1 点分治

算法 8.1 点分治

```
1: procedure Solve(n)
2:     Decompose(1, n)
3: procedure Decompose(u, whole)
4:     rt ← 0
5:     FindRoot(u, whole, rt, 0)
6:     vis[rt] ← true
7:     Calc(rt)
8:     for v ∈ G[rt] do
9:         if vis[v] then
10:            continue
11:        ss ← { siz[v],              if siz[v] < siz[rt],
                { whole − siz[rt],    otherwise.
12:        Decompose(v, ss)
13: procedure FindRoot(u, whole, out rt, fa)
14:    siz[u] ← 1
15:    mxs[u] ← 0
16:    for v ∈ G[u] do
17:        if vis[v] or v = fa then
18:            continue
19:        FindRoot(v, whole, rt, u)
20:        siz[u] ← siz[u] + siz[v]
21:        mxs[u] ← max(mxs[u], siz[v])
22:    mxs[u] ← max(mxs[u], whole − siz[u])
23:    if 2 · mxs[u] ≤ whole then
24:        rt ← u
25: procedure Calc(rt)
26:                                  ▷ problem-specific; do not modify vis, siz, mxs
```

8.1.1 待实现

`Calc(rt)` 统计经过 `rt` 的路径。按子树分批处理：先查询当前结构，再插入该子树，避免同子树点对。

8.1.2 数据契约

`siz` 以本次 `FindRoot` 的入口 `u` 为根，故 `ss` 要按 `siz[v] < siz[rt]` 分儿子 / 父亲，父亲一侧取 `whole - siz[rt]`。

`Decompose` 在 `Calc` 之后才用 `siz` 算 `ss`，因此 `Calc` 不得改写 `vis`、`siz`、`mxs`。

`FindRoot` 递归深度为连通块高度，链上可达 $O(n)$；`Decompose` 的递归深度才是 $O(\log n)$。

8.2 虚树

算法 8.2 虚树（单调栈）

```
1:  procedure Build(h, k)
2:      sort h[1..k] by dfn
3:      top ← 1
4:      sta[1] ← 1
5:      for i ← 1 to k do
6:          if h[i] = 1 then
7:              continue
8:          l ← LCA(h[i], sta[top])
9:          if l ≠ sta[top] then
10:             while dfn[l] < dfn[sta[top − 1]] do
11:                 AddEdge(sta[top − 1], sta[top])
12:                 top ← top − 1
13:             if dfn[l] > dfn[sta[top − 1]] then
14:                 AddEdge(l, sta[top])
15:                 sta[top] ← l
16:             else
17:                 AddEdge(l, sta[top])
18:                 top ← top − 1
19:         top ← top + 1
20:         sta[top] ← h[i]
21:     for i ← 1 to top − 1 do
22:         AddEdge(sta[i], sta[i + 1])
```

8.2.1 待实现

`Build` 依赖预处理好的 `dfn` 与 `LCA`。`AddEdge(u, v)` 加有向边 $u \to v$（父到子），有边权时取原树 `dist[v] - dist[u]`；建完后从根 1 开始跑 `Calc`，进入前标好关键点，结束后撤销。

8.2.2 数据契约

`h[1..k]` 是互异的关键点，1-indexed，原树根固定为 1；进入 `Build` 前未排序，返回时按 `dfn` 升序。

`sta[1..top]` 维护当前最右链，`dfn` 从栈底到栈顶递增，`sta[1] = 1`。弹栈条件比较 `l` 与次栈顶 `sta[top-1]`，不是栈顶；循环结束后剩下的 `sta[1..top]` 还要逐条连边。

8.3 Tarjan

8.3.1 边双连通分量

算法 8.3 边双连通分量（Tarjan）

```
1:  procedure Tarjan(u, in)
2:      dfc ← dfc + 1
3:      dfn[u] ← dfc
4:      low[u] ← dfc
5:      stk.Push(u)
6:      vis[u] ← true
7:      for i ∈ G[u] do
8:          if i = in ⊕ 1 then
9:              continue
10:         v ← to[i]
11:         if dfn[v] = 0 then
12:             Tarjan(v, i)
13:             low[u] ← min(low[u], low[v])
14:         else if vis[v] then
15:             low[u] ← min(low[u], dfn[v])
16:     if dfn[u] = low[u] then
17:         C ← ∅
18:         repeat
19:             x ← stk.Pop()
20:             vis[x] ← false
21:             C.Add(x)
22:         until x = u
```

8.3.2 点双连通分量

算法 8.4 点双连通分量（Tarjan）

```
1:  procedure Tarjan(u)
2:      dfc ← dfc + 1
3:      dfn[u] ← dfc
4:      low[u] ← dfc
5:      stk.Push(u)
6:      if u = root and G[u] = ∅ then
7:          C ← ∅
8:          C.Add(u)
9:          return
10:     f ← 0
11:     for v ∈ G[u] do
12:         if dfn[v] = 0 then
13:             Tarjan(v)
14:             low[u] ← min(low[u], low[v])
15:             if low[v] ≥ dfn[u] then
16:                 f ← f + 1
17:                 if u ≠ root or f > 1 then
18:                     cut[u] ← true
19:                 C ← ∅
20:                 repeat
21:                     x ← stk.Pop()
22:                     C.Add(x)
23:                 until x = v
24:                 C.Add(u)
25:         else
26:             low[u] ← min(low[u], dfn[v])
```

8.3.3 圆方树

算法 8.5 圆方树

```
1:  procedure Tarjan(u)
2:      dfc ← dfc + 1
3:      dfn[u] ← dfc
4:      low[u] ← dfc
5:      stk.Push(u)
6:      for v ∈ G[u] do
7:          if dfn[v] = 0 then
8:              Tarjan(v)
9:              low[u] ← min(low[u], low[v])
10:             if low[v] = dfn[u] then
11:                 cnt ← cnt + 1
12:                 create square vertex cnt
13:                 repeat
14:                     x ← stk.Pop()
15:                     AddEdge(cnt, x)
16:                 until x = v
17:                 AddEdge(cnt, u)
18:         else
19:             low[u] ← min(low[u], dfn[v])
```

8.3.4 数据契约

边双连通分量：`G[u]` 存弧编号，`to[i]` 是弧头；`in` 是进入 `u` 的弧编号，只跳过 `in ^ 1`，不能按 `fa` 跳过，重边是合法返祖边。`vis` 只表示点还在栈中，已出栈的点不能再更新 `low`。边池从 2 开始编号，1 号空着，无向边按 `2k / 2k+1` 成对加入，保证 `in = 0` 时 `in ^ 1 = 1` 不对应任何弧；自环要先删掉。

点双连通分量：满足 `low[v] >= dfn[u]` 时弹栈到 `v`，再把 `u` 加入该点双；`u` 不弹，它还可能属于父方向的点双。`f` 只数满足该条件的分支：非根点第一次就标割点，根要 `f > 1`。孤立根没有分支，要单独产出一个点双；已访问邻居用 `dfn[v]` 更新 `low`，只有树边递归返回后才用 `low[v]`。

圆方树：`cnt` 从 `n` 开始，方点编号是 `n+1..`，数组要开两倍。每个点双建一个方点，向分量内所有圆点连无向边；桥也形成一个二元点双，所以桥两端会通过方点相连。`u` 只连方点，不从栈中弹出；孤立点没有点双，保持为单独的圆点。点双与圆方树的 `low` 计入父边；圆方树的判定用 `low[v] == dfn[u]`，若 `low` 不含父边则改用 `low[v] >= dfn[u]`，两种写法不要混用。

图不连通时，三者都要对每个未访问点重新调用：边双传 `in = 0`，点双与圆方树重设 `root`。

8.4 杜教筛

8.4.1 作用与适用范围

杜教筛对任意数论函数 f 求前缀和 $S(n)=\sum_{i=1}^{n} f(i)$。适用于 n 大到不能预处理 $1..n$ 的情形。递归只依赖 $O(\sqrt{n})$ 个不同的 $\lfloor n/i \rfloor$，配合记忆化可做到亚线性。线性筛预处理前 $B \approx n^{2/3}$ 项后复杂度为 $O(n^{2/3})$，不做预处理为 $O(n^{3/4})$。

它不要求 f 是积性函数；需要构造数论函数 g（$f*g$ 为狄利克雷卷积），使

- $\sum_{i=1}^{n}(f*g)(i)$ 能快速求值；
- $\sum_{i=1}^{n} g(i)$ 能快速求值，且 $g(1)$ 非零（模意义下可逆）。

8.4.2 算法简述

按

$$g(1)S(n)=\sum_{i=1}^{n}(f*g)(i)-\sum_{i=2}^{n} g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

递归，右侧用数论分块，S 记忆化。通常取 $g(1)=1$；若 $g(1)\neq 1$，最后除以 $g(1)$。常见构造是 $\mu*1=\epsilon$、$\varphi*1=\mathrm{id}$；例如 $f=\mathrm{id}^2\,\varphi$ 时取 $g=\mathrm{id}^2$，则 $f*g=\mathrm{id}^3$。

8.4.3 伪代码

记 $\mathrm{pre}[n]$ 为线性筛预处理的 $S(n)$，mem 为记忆化表：

算法 8.6 杜教筛

```
1:  procedure SumFG(n)
2:      return Σ_{i=1}^{n} (f*g)(i)
3:  procedure SumG(n)
4:      return Σ_{i=1}^{n} g(i)
5:  procedure Sum(n)
6:      if n ≤ B then
7:          return pre[n]
8:      if n ∈ mem then
9:          return mem[n]
10:     ans ← SumFG(n)
11:     l ← 2
12:     while l ≤ n do
13:         r ← n/(n/l)
14:         ans ← ans − (SumG(r) − SumG(l − 1)) · Sum(n/l)
15:         l ← r + 1
16:     mem[n] ← ans/g(1)
17:     return mem[n]
```

8.5 Min_25 筛

8.5.1 作用与适用范围

Min_25 筛求积性函数 f 的前缀和 $\sum_{i=1}^{n} f(i)$，适合杜教筛难以构造 g、而 $f(p)$ 是低次多项式或若干完全积性函数之和的情形，n 通常到 10^{10} 量级；复杂度为 $O(n^{3/4}/\log n)$ 或 $O(n^{1-\epsilon})$。

它要求：

- f 是积性函数，且 $f(1)=1$；
- $f(p)$ 可写成 $\sum_j a_j p^{c_j}$（或若干完全积性函数在质数处的线性组合），项数较少；
- $f(p^c)$ 能快速计算；
- 能筛出 $\sqrt{n}$ 内的质数，并能快速求各辅助完全积性函数 g 的整数前缀和 $\sum_{1\le t\le x} g(t)$。特别地，当 $g(t)=t^e$ 且 e 为非负整数时，低次幂和可用公式计算，高次幂和可用 Lagrange 插值计算。

8.5.2 算法简述

记 $p_1,p_2,\dots$ 为质数，$F_{\text{prime}}(x)=\sum_{p\le x} f(p)$，$F_k(x)$ 为 $2\le t\le x$ 且最小质因子不小于 p_k 的 $f(t)$ 之和（$p_0=1$）。当 $x<p_k$ 时，约定 $F_k(x)=0$。先对 $f(p)$ 的每个单项 $g(p)=p^e$ 做埃氏筛式递推

$$G_k(x)=G_{k-1}(x)-p_k^e\left(G_{k-1}\left(\left\lfloor\frac{x}{p_k}\right\rfloor\right)-G_{k-1}(p_{k-1})\right),\qquad p_k^2\le x,$$

其中 $G_0(x)=\sum_{2\le t\le x} t^e$，$G_k(x)$ 是筛掉最小质因子不大于 p_k 的合数后，剩余整数的 g 值之和；当 $p_k^2>x$ 时，$G_k(x)=G_{k-1}(x)$。由于 $G_{k-1}(p_k-1)=G_{k-1}(p_{k-1})$，递推时也可使用前者。合并各单项，得到所有 $x=\lfloor n/i\rfloor$ 处的 $F_{\text{prime}}(x)$，再对 $x\ge p_k$ 按

$$F_k(x)=F_{\text{prime}}(x)-F_{\text{prime}}(p_{k-1})+\sum_{\substack{i\ge k\\ p_i^2\le x}}\sum_{\substack{c\ge 1\\ p_i^c\le x}} f(p_i^c)\left([c>1]+F_{i+1}\left(\left\lfloor\frac{x}{p_i^c}\right\rfloor\right)\right)$$

递归，答案为 $1+F_1(n)$。例如 $f(p^c)=p^c(p^c-1)$ 时 $f(p)=p^2-p$，拆成 p^2 与 $-p$ 两个单项即可。

8.5.3 伪代码

下面用 $w_1>\cdots>w_m$ 表示所有不同的 $\lfloor n/i\rfloor$，$\mathrm{id}(x)$ 为 x 在其中的下标，a_e 为单项 p^e 的系数，r 为不超过 $\sqrt{n}$ 的质数个数。Fprime、id 和质数表由 BuildFprime(n) 构造，并供 Calc(x,k) 使用；伪代码中的除法均取下整。

算法 8.7 Min_25 筛

procedure BuildFprime(n)
  let $p_1, p_2, \ldots, p_r$ **be all primes** $\le \lfloor\sqrt{n}\rfloor$, $p_0 = 1$
  let $w_1 > \cdots > w_m$ **be all distinct** $\lfloor n/i \rfloor$, **with** $\mathrm{id}(w_i) = i$
  $\mathrm{Fprime}[1..m] \leftarrow 0$
  for each monomial $g(p) = p^e$ **with coefficient** a_e **do**
    for $i \leftarrow 1$ **to** m **do**
      $G[i] \leftarrow \sum_{2 \le t \le w_i} t^e$
    for $j \leftarrow 1$ **to** r **do**
      $p \leftarrow p_j$
      for $i \leftarrow 1$ **to** m **with** $p^2 \le w_i$ **do**
        $G[i] \leftarrow G[i] - p^e \left(G[\mathrm{id}(\lfloor w_i/p \rfloor)] - G[\mathrm{id}(p-1)]\right)$
    $\mathrm{Fprime}[1..m] \leftarrow \mathrm{Fprime}[1..m] + a_e G[1..m]$
procedure Calc(x, k)
  if $x \le p_{k-1}$ **then**
    return 0
  $\mathrm{ans} \leftarrow \mathrm{Fprime}[\mathrm{id}(x)] - \mathrm{Fprime}[\mathrm{id}(p_{k-1})]$
  $i \leftarrow k$
  while $i \le r$ **and** $p_i^2 \le x$ **do**
    $\mathrm{pe} \leftarrow p_i$
    $c \leftarrow 1$
    while $\mathrm{pe} \le x$ **do**
      $\mathrm{ans} \leftarrow \mathrm{ans} + f(\mathrm{pe}) \left([c > 1] + \mathrm{Calc}(\lfloor x/\mathrm{pe} \rfloor, i+1)\right)$
      $\mathrm{pe} \leftarrow \mathrm{pe} \cdot p_i$
      $c \leftarrow c + 1$
    $i \leftarrow i + 1$
  return ans

调用 BuildFprime(n) 后，最终答案为 $1 + \mathrm{Calc}(n, 1)$。

8.6 PN 筛

8.6.1 作用与适用范围

PN 筛对积性函数 f 求前缀和 $F(n) = \sum_{i=1}^{n} f(i)$。它利用 Powerful Number（所有质因子指数均 ≥ 2 的数；计算时把 1 也计入）把求和压到 $O(\sqrt{n})$ 个 PN 上，适合 $f(p)$ 形式简单、$f(p^c)$ 可快速计算，且能构造出易求前缀和的 g 的情形。

它要求：

- f 是积性函数；
- 存在积性函数 g，满足 $g(p) = f(p)$（对所有质数 p），且 $G(n) = \sum_{i=1}^{n} g(i)$ 易求（$O(1)$ 或杜教筛）；

- $f(p^c)$ 能快速计算；
- 能筛出 $\sqrt{n}$ 内的质数，用于 DFS 枚举 PN。

8.6.2 算法简述

令 $h = f/g$（狄利克雷卷积除法，即 $f = g * h$），则 h 积性、$h(1) = 1$、$h(p) = 0$，且 h 只在 PN 处非零。于是

$$F(n) = \sum_{\substack{d \le n \\ d \text{ 是 PN}}} h(d)\, G\left(\left\lfloor \frac{n}{d} \right\rfloor\right).$$

其中 $d = 1$ 也计入。

$h(p^c)$ 按

$$h(p^c) = f(p^c) - \sum_{i=1}^{c} g(p^i) h(p^{c-i})$$

递推（$h(p) = 0$）。按质因数指数 DFS 枚举所有 PN（每个 PN 可写成 $a^2 b^3$），边搜边累加；n 内 PN 只有 $O(\sqrt{n})$ 个。若 G 用杜教筛，总复杂度为 $O(n^{2/3})$。

8.6.3 伪代码

设 $p_1, p_2, \ldots$ 为质数，‘Dfs(d, k, hd)‘ 中 d 是当前 PN，k 是下一个可用的最小质因子下标，$hd = h(d)$。循环只枚举 $i \ge k$ 且 $p_i^2 \le n/d$ 的质数，保证当前 d 还能再乘上 p_i^2；对每个这样的质数，先按递推求出 $h(p_i^0), \ldots, h(p_i^C)$，再枚举指数 $c \ge 2$：

算法 8.8 PN 筛

```
1: procedure CalcH(p, C)
2:     hp[0] ← 1
3:     hp[1] ← 0
4:     for c ← 2 to C do
5:         hp[c] ← f(p^c)
6:         for i ← 1 to c do
7:             hp[c] ← hp[c] − g(p^i) · hp[c − i]
8:     return hp[0..C]
9: procedure SumG(n)
10:    return ∑_{i=1}^{n} g(i)
11: procedure Dfs(d, k, hd)
12:    ans ← ans + hd · SumG(n/d)
13:    for each i ≥ k with p_i^2 ≤ n/d do
14:        C ← max{c ≥ 2 : d · p_i^c ≤ n}
15:        hp ← CalcH(p_i, C)
16:        for c ← 2 to C do
17:            Dfs(d · p_i^c, i + 1, hd · hp[c])
18: procedure Calc(n)
19:    ans ← 0
20:    Dfs(1, 1, 1)
21:    return ans
```